claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_b7f012f8-5c3\agent-ab53bd3145cc00d1b.jsonl`
- SHA-256: `c6f81afdbbd88644b0d9133fe7973873ae4bcde900eecbf5a0e1b135f99d51a6`
- Source modified: `2026-06-18T01:31:45+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_b7f012f8-5c3`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T01:27:48.618Z)

```
You are an adversarial bug-hunter in a mature Python repo (badminton-highlight-indexer:
FastAPI + SQLite + ffmpeg + GPU CV/AI rally detector). Read code READ-ONLY in this worktree
(pinned to origin/master ? DO NOT modify anything):
  C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base

Hunt ONLY for REAL, REACHABLE defects a maintainer would agree is a bug and want fixed:
- crash on a reachable input (unguarded dict/index/attr access, None deref,
KeyError/IndexError),
- wrong logic / off-by-one / inverted condition / wrong operator,
- incorrect error handling (catching too much/little, swallowing, wrong exception type),
- resource leak (unclosed file/db/subprocess), TOCTOU, mutable-default-arg,
- contract mismatch between a function and its callers/tests,
- wrong default / unit / sign that produces incorrect output.

HARD RULES:
- PRECISION over recall. Report a finding ONLY if you traced the actual code and a concrete
  trigger reaches it. No style, no "could be cleaner", no speculative "might", no perf-nits,
  no missing-test observations, no naming. If you are not sure it is a real bug, DROP it.
- For each finding give an EXACT file path + line, the bug_class, a CONCRETE trigger (the input
  or call path that hits it), why it is wrong, severity, and a minimal proposed fix.
- This repo has 848 passing tests and several hardening sweeps ? easy bugs are already gone.
  Expect FEW or ZERO real bugs in your area. Returning an empty candidates list is a VALID,
  respected answer. Do NOT manufacture findings to look productive.
- EXCLUDE these files entirely (just fixed / fix already in flight): backend/eval/ablation.py,
  backend/eval/fusion_compare.py, backend/eval/fusion_audio.py.
- Verify your trigger by reading the surrounding code and any related test, not from memory.

YOUR AREA: backend/infrastructure/,
backend/config/, backend/utils/, backend/features/. Focus on the SQLite migration ladder
(PRAGMA user_version), DB connection/transaction handling, Pydantic config validation/defaults,
hashing (compute_video_id), and run_telemetry/video helpers.
```

2. assistant (2026-06-18T01:27:50.866Z)

```
I'll hunt for bugs in my assigned area. Let me start by pulling the latest and exploring the
target directories.
```

3. user (2026-06-18T01:27:52.684Z)

```
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
Not currently on any branch.
nothing to commit, working tree clean
```

4. user (2026-06-18T01:27:53.021Z)

```
backend\infrastructure\execution_policy.py
backend\infrastructure\database.py
```

5. user (2026-06-18T01:27:53.499Z)

```
backend\config\__init__.py
backend\config\loader.py
backend\config\models.py
```

6. user (2026-06-18T01:27:53.981Z)

```
backend\utils\__init__.py
backend\utils\geometry.py
backend\utils\paths.py
backend\utils\video.py
backend\utils\logging_setup.py
backend\utils\run_telemetry.py
backend\utils\hashing.py
```

7. user (2026-06-18T01:27:54.084Z)

```
backend\features\__init__.py
backend\features\rally_seq.py
```

8. assistant (2026-06-18T01:27:58.665Z)

Let me read all the files in my area. Starting with the largest/highest-risk ones.

9. user (2026-06-18T01:28:00.369Z)

```
1       import sqlite3
2       import json
3       import logging
4       from contextlib import contextmanager
5       from typing import List, Dict, Any, Optional, Tuple
6
7       logger = logging.getLogger(__name__)
8
9       class Database:
10          def __init__(self, db_path: str):
11              self.db_path = db_path
12              self._init_db()
13
14          @contextmanager
15          def _connect(self):
16              """Transaction scope that also CLOSES the connection.
17
18              ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19              never closes, so every call leaked its connection to GC (lingering file
20              handles on the WAL sidecars under Windows). Internal methods use this.
21              """
22              conn = self._get_connection()
23              try:
24                  with conn:
25                      yield conn
26              finally:
27                  conn.close()
28
29          def _get_connection(self):
30              # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31              # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
32              conn = sqlite3.connect(self.db_path, timeout=30.0)
33              conn.row_factory = sqlite3.Row
```

```
            # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
            conn.execute("PRAGMA foreign_keys = ON")
            # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
            conn.execute("PRAGMA busy_timeout = 30000")
            try:
                conn.execute("PRAGMA journal_mode = WAL")
            except sqlite3.OperationalError:
                pass  # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
            return conn

        def _init_db(self):
            """Initializes tables for videos, play_segments, and events."""
            with self._connect() as conn:
                cursor = conn.cursor()

                # Videos Table
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS videos (
                        id TEXT PRIMARY KEY,
                        filepath TEXT NOT NULL,
                        processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                        status TEXT NOT NULL,
                        validation_results TEXT
                    )
                """)

                # Play Segments Table (Extensible metadata JSON)
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS play_segments (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        video_id TEXT NOT NULL,
                        start_time REAL NOT NULL,
                        end_time REAL NOT NULL,
                        duration REAL NOT NULL,
                        server TEXT,
                        receiver TEXT,
                        metadata TEXT,
                        FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
                    )
                """)

                # Events Table
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS events (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        segment_id INTEGER NOT NULL,
                        timestamp REAL NOT NULL,
                        type TEXT NOT NULL,
                        player TEXT,
                        details TEXT,
                        FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
CASCADE
                    )
                """)

                # Versioned migrations live here. PRAGMA user_version is the schema
                # contract ? bump it for every change and add an ordered `if version < N`
                # block below. Tests assert the expected version, so accidental drift fails
CI.
                version = cursor.execute("PRAGMA user_version").fetchone()[0]

                if version < 1:
                    # v1: raw candidate detections (pre-confirmation), detector-agnostic.
                    # Common fields are columns; detector-specific metrics go in `stats`
JSON,
```

```
                    # so a new segmenter reuses this table by setting its own
`detector`/`stats`.
                    cursor.execute("""
                        CREATE TABLE IF NOT EXISTS candidate_segments (
                            id INTEGER PRIMARY KEY AUTOINCREMENT,
                            video_id TEXT NOT NULL,
                            detector TEXT NOT NULL,
                            chunk_index INTEGER,
                            start_time REAL NOT NULL,
                            end_time REAL NOT NULL,
                            duration REAL NOT NULL,
                            confidence REAL,
                            stats TEXT,
                            detection_params TEXT,
                            confirmed_segment_id INTEGER,
                            human_label TEXT,
                            notes TEXT,
                            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                            FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
                            FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
                        )
                    """)
                    cursor.execute("""
                        CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
                            ON candidate_segments(video_id, detector)
                    """)
                    cursor.execute("PRAGMA user_version = 1")

                if version < 2:
                    # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per
submitted
                    # /api/process request. `status` is the EXECUTION state of the job
                    # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
                    # that failed validation) lives inside `result` JSON as
result["status"].
                    # `result` is the full sync-era response dict (incl. report paths), so
the
                    # observability report is the job's artifact, per the plan.
                    cursor.execute("""
                        CREATE TABLE IF NOT EXISTS jobs (
                            id TEXT PRIMARY KEY,
                            video_path TEXT NOT NULL,
                            video_id TEXT,
                            segmenter TEXT,
                            player_pool TEXT,
                            max_frames INTEGER,
                            status TEXT NOT NULL DEFAULT 'queued',
                            progress TEXT,
                            error TEXT,
                            result TEXT,
                            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                            started_at TIMESTAMP,
                            finished_at TIMESTAMP
                        )
                    """)
                    cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON
jobs(status)")
                    cursor.execute("PRAGMA user_version = 2")

                if version < 3:
                    # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
`policy` = the
                    # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
job is due
```

```
153                     # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
any worker
154                     # re-claims it via `claim_due_job` when due ? with NO master node: the
table IS
155                     # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
156                     cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
157                     cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
158                     cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
159                     cursor.execute("PRAGMA user_version = 3")
160
161                 if version < 4:
162                     # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a
NULLABLE
163                     # `user_id` to videos + candidate_segments so a future per-user / multi-
tenant
164                     # path can scope rows to an owner. **NULL = local install = byte-
identical to
165                     # today** ? every existing row stays NULL and every existing query
(which never
166                     # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive.
167                     # Additive ONLY: no served-behavior change rides on this slice.
168                     cursor.execute("ALTER TABLE videos ADD COLUMN user_id TEXT")
169                     cursor.execute("ALTER TABLE candidate_segments ADD COLUMN user_id TEXT")
170                     # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast
path free
171                     # of index churn while making the eventual per-user lookups cheap.
172                     cursor.execute(
173                         "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
174                         "WHERE user_id IS NOT NULL")
175                     cursor.execute(
176                         "CREATE INDEX IF NOT EXISTS idx_candidates_user "
177                         "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
178                     cursor.execute("PRAGMA user_version = 4")
179
180                 if version < 5:
181                     # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per
(user_id,
182                     # version): the resolved per-user `fusion_config` overlay + an INLINE
183                     # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
184                     # and provenance that earned it. `is_active` pins the one row the
serving bridge
185                     # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`).
This slice
186                     # only persists/loads it ? NOTHING reads it on the served path yet.
187                     #
188                     #   user_id              ? owner of the model (NULL allowed = the local
install)
189                     #   version              ? monotonic per-user model version
190                     #   baseline_version     ? the shared baseline this was fit against
(upgrade switch)
191                     #   feature_schema_hash  ? fingerprint of the feature set; MINOR-vs-
MAJOR re-fit gate
192                     #   fusion_config        ? JSON per-user FusionConfig overlay (cue_flags
/ thresholds)
193                     #   classifier_json      ? JSON LogisticRegression.to_dict() (None =
config-only model)
194                     #   promotion_evidence   ? JSON per_user_gate PromotionDecision (why it
was promoted)
195                     #   n_videos_at_fit      ? held-out-user corpus size when fit (min_n
trust floor input)
196                     #   is_active            ? the one resolved row for this user (partial-
unique below)
```

```
                    cursor.execute("""
                        CREATE TABLE IF NOT EXISTS user_models (
                            id INTEGER PRIMARY KEY AUTOINCREMENT,
                            user_id TEXT,
                            version INTEGER NOT NULL DEFAULT 1,
                            baseline_version TEXT,
                            feature_schema_hash TEXT,
                            fusion_config TEXT,
                            classifier_json TEXT,
                            promotion_evidence TEXT,
                            n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
                            is_active INTEGER NOT NULL DEFAULT 0,
                            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                        )
                    """)
                    # One model version per user (NULLs compare distinct in SQLite, so the
local
                    # install can still carry multiple versioned rows; the active-row guard
below
                    # is what enforces a single served model).
                    cursor.execute("""
                        CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
                            ON user_models(user_id, version)
                    """)
                    # At most ONE active model per user: a partial-unique index over the
active rows.
                    # (SQLite treats NULL user_id rows as distinct, so the local install's
single
                    # active row is still guarded ? there is one local user.)
                    cursor.execute("""
                        CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
                            ON user_models(user_id) WHERE is_active = 1
                    """)
                    cursor.execute("PRAGMA user_version = 5")
                # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 6

                conn.commit()

        def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
            """Register or update a video row WITHOUT touching its child segments.

            Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
            with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
            That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
            before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
            row in place; clearing segments is now an explicit, deliberate operation
            (clear_video_data / prune_segments).
            """
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "INSERT INTO videos (id, filepath, status, validation_results)
VALUES (?, ?, ?, ?) "
                        "ON CONFLICT(id) DO UPDATE SET "
                        "filepath=excluded.filepath, status=excluded.status, "
                        "validation_results=excluded.validation_results",
                        (video_id, filepath, status, json.dumps(validation_results))
                    )
                    conn.commit()
                return True
```

```
            except Exception as e:
                logger.error(f"Failed to add video: {e}")
                return False

        def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
            """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).

            Captured before a (re)segmentation so the run's new rows (id > watermark) can be
            told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
            """
            with self._connect() as conn:
                cur = conn.cursor()
                p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
                                (video_id,)).fetchone()[0]
                c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
                                (video_id,)).fetchone()[0]
                return int(p), int(c)

        def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
                           play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
                           cand_after: Optional[int] = None) -> None:
            """Delete play/candidate segments by id range (events cascade from
play_segments).

            Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
            successful run (``*_upto`` = the pre-run watermark), or drop the NEW partial
rows
            and keep the OLD ones when the run failed/produced nothing (``*_after``).
            """
            with self._connect() as conn:
                cur = conn.cursor()
                if play_upto is not None:
                    cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
                if play_after is not None:
                    cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
                if cand_upto is not None:
                    cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
                if cand_after is not None:
                    cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
                conn.commit()

        def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
            try:
                with self._connect() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
                        (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add play segment: {e}")
```

```
                return None

        def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
                        (segment_id, timestamp, event_type, player, json.dumps(details or
{}))
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to add event: {e}")
                return False

        def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
                                  chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
                                  stats: Optional[Dict[str, Any]] = None,
                                  detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
            """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
            detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
            try:
                with self._connect() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        """INSERT INTO candidate_segments
                           (video_id, detector, chunk_index, start_time, end_time, duration,
                            confidence, stats, detection_params)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
                        (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
                         confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add candidate segment: {e}")
                return None

        def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
            """Record that a candidate detection was confirmed as a given play_segment."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
?",
                        (segment_id, candidate_id)
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to link candidate {candidate_id} to segment
{segment_id}: {e}")
                return False

        def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
                                     only_unlabeled: bool = False) -> List[Dict[str, Any]]:
```

```
            """Fetch candidate detections for the annotation / fine-tuning workflow.
            `stats` and `detection_params` are deserialized from JSON."""
            query = "SELECT * FROM candidate_segments WHERE video_id = ?"
            params: List[Any] = [video_id]

            if detector:
                query += " AND detector = ?"
                params.append(detector)
            if only_unlabeled:
                query += " AND human_label IS NULL"

            query += " ORDER BY start_time"

            candidates = []
            with self._connect() as conn:
                rows = conn.cursor().execute(query, params).fetchall()
                for row in rows:
                    d = dict(row)
                    d["stats"] = json.loads(d["stats"] or "{}")
                    d["detection_params"] = json.loads(d["detection_params"] or "{}")
                    candidates.append(d)
            return candidates

        def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
            with self._connect() as conn:
                row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
                if row:
                    d = dict(row)
                    d["validation_results"] = json.loads(d["validation_results"] or "[]")
                    return d
            return None

        def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
            """
            Dynamically queries play segments based on filters:
            - duration_min: float
            - server: string
            - receiver: string
            - event_type: string (e.g. smash, foul)
            """
            query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
            params = [video_id]

            if filters.get("duration_min"):
                query += " AND r.duration >= ?"
                params.append(filters["duration_min"])

            if filters.get("server"):
                query += " AND r.server = ?"
                params.append(filters["server"])

            if filters.get("receiver"):
                query += " AND r.receiver = ?"
                params.append(filters["receiver"])

            if filters.get("event_type"):
                # Check if there is an event of a specific type inside this segment
                query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
                params.append(filters["event_type"])

            segments_list = []
            with self._connect() as conn:
```

```
                cursor = conn.cursor()
                rows = cursor.execute(query, params).fetchall()

                for row in rows:
                    r_dict = dict(row)
                    r_id = r_dict["id"]
                    r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")

                    # Fetch associated events
                    event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
                    r_dict["events"] = []
                    for er in event_rows:
                        e_dict = dict(er)
                        e_dict["details"] = json.loads(e_dict["details"] or "{}")
                        r_dict["events"].append(e_dict)

                    segments_list.append(r_dict)

            return segments_list

        def clear_video_data(self, video_id: str):
            with self._connect() as conn:
                conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
                conn.commit()

        # --- Async job store (DEFERRED_HARDENING_PLAN #16) ------------------------------
        # Writers follow the repo convention (swallow + logger.error, return bool); the
        # worker logs loudly on a failed transition so a job can't silently wedge.

        def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
                       player_pool: Optional[List[str]] = None,
                       max_frames: Optional[int] = None,
                       policy: Optional[Dict[str, Any]] = None) -> bool:
            """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
            (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "INSERT INTO jobs (id, video_path, segmenter, player_pool,
max_frames, status, policy) "
                        "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
                        (job_id, video_path, segmenter,
                         json.dumps(player_pool) if player_pool is not None else None,
                         max_frames, json.dumps(policy) if policy is not None else None)
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to create job {job_id}: {e}")
                return False

        def mark_job_running(self, job_id: str) -> bool:
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
                        "progress='starting' WHERE id = ?", (job_id,))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to mark job {job_id} running: {e}")
                return False

```

```
        def set_job_progress(self, job_id: str, progress: str) -> bool:
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to set progress for job {job_id}: {e}")
                return False

        def set_job_video_id(self, job_id: str, video_id: str) -> bool:
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to set video_id for job {job_id}: {e}")
                return False

        def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
            """Job EXECUTED to completion. The pipeline's own verdict (success/failed
            validation) is inside `result` ? job status 'done' means 'the worker
finished'."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET status='done', progress='finished', result=?, "
                        "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
                        (json.dumps(result), job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to mark job {job_id} done: {e}")
                return False

        def mark_job_failed(self, job_id: str, error: str) -> bool:
            """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET status='failed', error=?, "
                        "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to mark job {job_id} failed: {e}")
                return False

        def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
            """Fetch one job; `result`/`player_pool` JSON deserialized."""
            with self._connect() as conn:
                row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
(job_id,)).fetchone()
                if not row:
                    return None
                d = dict(row)
                d["result"] = json.loads(d["result"]) if d["result"] else None
                d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
None
                d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
                return d

```

```
        # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
------------------------
        def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
            """Attach/replace a job's JSON ExecutionPolicy."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET policy=? WHERE id = ?",
                        (json.dumps(policy) if policy is not None else None, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to set policy for job {job_id}: {e}")
                return False

        def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
None) -> bool:
            """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
            worker. `claim_due_job` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
            computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET status='waiting', "
                        "next_poll_at=datetime('now', ?), "
                        "progress=COALESCE(?, progress) WHERE id = ?",
                        (f"+{max(0, int(delay_sec))} seconds", progress, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to set job {job_id} waiting: {e}")
                return False

        def seconds_until_next_due(self) -> Optional[float]:
            """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
            schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
            already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
            try:
                with self._connect() as conn:
                    row = conn.cursor().execute(
                        "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
                        "  (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
secs "
                        "FROM jobs WHERE status='waiting'").fetchone()
                if row is None or row["secs"] is None:
                    return None
                return max(0.0, float(row["secs"]))
            except Exception as e:
                logger.error(f"Failed to compute next-due delay: {e}")
                return None

        def claim_due_job(self) -> Optional[Dict[str, Any]]:
            """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
            flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
            double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
            first. Returns the claimed job dict, or None if nothing is runnable."""
```

```
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    row = cur.execute(
                        "SELECT id, status FROM jobs WHERE status='queued' "
                        "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
                        "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
                    if not row:
                        return None
                    cur.execute(
                        "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
                        "WHERE id=? AND status=?", (row["id"], row["status"]))
                    conn.commit()
                    if cur.rowcount != 1:
                        return None                    # lost the race to a concurrent
worker
                return self.get_job(row["id"])
            except Exception as e:
                logger.error(f"Failed to claim a due job: {e}")
                return None

        def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
            """Crash recovery: any job still queued/running from a previous process is dead
            (the executor is in-process). Mark failed so clients see a terminal state, not a
            hang. Called once at startup. Returns the number of jobs swept."""
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    cur.execute(
                        "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
                        "WHERE status IN ('queued', 'running')", (reason,))
                    conn.commit()
                    return cur.rowcount
            except Exception as e:
                logger.error(f"Failed to sweep stale jobs: {e}")
                return 0

        # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) ------------------------
        # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
        # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
        # serving.py) resolves them, and wiring that bridge into the production decision
seam
        # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
        # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
        # JSON-serialised dicts; the rest are scalar columns.

        def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,
                              baseline_version: Optional[str] = None,
                              feature_schema_hash: Optional[str] = None,
                              fusion_config: Optional[Dict[str, Any]] = None,
                              classifier_json: Optional[Dict[str, Any]] = None,
                              promotion_evidence: Optional[Dict[str, Any]] = None,
                              n_videos_at_fit: int = 0,
                              is_active: bool = False) -> Optional[int]:
            """Insert or replace ONE per-user model row keyed by ``(user_id, version)``.

            Idempotent on that key (UPSERT, not INSERT-OR-REPLACE: REPLACE would re-allocate
the
            rowid). When ``is_active`` is True this row becomes the user's single active
model ?
            any previously-active row for the SAME user is first cleared, so the
            ``idx_user_models_one_active`` partial-unique invariant always holds (one active
```

```
model
            per user). Returns the row id, or None on failure.
            """
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    if is_active:
                        # Demote the current active row first (NULL user_id = local: IS NULL
match).
                        if user_id is None:
                            cur.execute(
                                "UPDATE user_models SET is_active = 0 "
                                "WHERE user_id IS NULL AND is_active = 1 AND version != ?",
                                (version,))
                        else:
                            cur.execute(
                                "UPDATE user_models SET is_active = 0 "
                                "WHERE user_id = ? AND is_active = 1 AND version != ?",
                                (user_id, version))
                    cur.execute(
                        """INSERT INTO user_models
                           (user_id, version, baseline_version, feature_schema_hash,
                            fusion_config, classifier_json, promotion_evidence,
                            n_videos_at_fit, is_active)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                           ON CONFLICT(user_id, version) DO UPDATE SET
                             baseline_version=excluded.baseline_version,
                             feature_schema_hash=excluded.feature_schema_hash,
                             fusion_config=excluded.fusion_config,
                             classifier_json=excluded.classifier_json,
                             promotion_evidence=excluded.promotion_evidence,
                             n_videos_at_fit=excluded.n_videos_at_fit,
                             is_active=excluded.is_active""",
                        (user_id, version, baseline_version, feature_schema_hash,
                         json.dumps(fusion_config) if fusion_config is not None else None,
                         json.dumps(classifier_json) if classifier_json is not None else
None,
                         json.dumps(promotion_evidence) if promotion_evidence is not None
else None,
                         n_videos_at_fit, 1 if is_active else 0))
                    conn.commit()
                    rid = cur.execute(
                        "SELECT id FROM user_models WHERE "
                        + ("user_id IS NULL" if user_id is None else "user_id = ?")
                        + " AND version = ?",
                        ((version,) if user_id is None else (user_id, version))).fetchone()
                    return int(rid[0]) if rid else None
            except Exception as e:
                logger.error(f"Failed to upsert user_model (user={user_id}, v={version}):
{e}")
                return None

        def get_active_user_model(self, user_id: Optional[str]) -> Optional[Dict[str, Any]]:
            """Return the single active per-user model row for ``user_id`` (NULL = local
install),
            with ``fusion_config`` / ``classifier_json`` / ``promotion_evidence`` JSON
deserialised,
            or None if the user has no active model. Does not touch the served pipeline."""
            with self._connect() as conn:
                if user_id is None:
                    row = conn.cursor().execute(
                        "SELECT * FROM user_models WHERE user_id IS NULL AND is_active =
1").fetchone()
                else:
                    row = conn.cursor().execute(
```

```
                        "SELECT * FROM user_models WHERE user_id = ? AND is_active = 1",
                        (user_id,)).fetchone()
                if not row:
                    return None
                d = dict(row)
                for col in ("fusion_config", "classifier_json", "promotion_evidence"):
                    d[col] = json.loads(d[col]) if d[col] else None
                return d

```

10. user (2026-06-18T01:28:00.139Z)

```
      """Canonical file-identity hashing for the indexer.

      `video_id` is the MD5 of the *whole* video file. Historically each entrypoint
      (`backend/main.py`, `run_eval.py`, `scripts/run_phase3_eval.py`,
`scripts/run_process_and_stitch.py`)
      defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
      The hash values agreed (same bytes hashed), but the duplicated code was a footgun:
      any future change to one copy would silently diverge the DB key.

      This module is the single source of truth. Import `compute_video_id` everywhere a
      video needs to be keyed.
      """

      from __future__ import annotations

      import hashlib

      # 64 KB matches the value used by the live pipeline (backend/main.py) before this
      # was centralized. The chunk size does not affect the resulting digest ? it only
      # bounds memory use while streaming large files.
      _CHUNK_SIZE = 65536


      def compute_video_id(filepath: str) -> str:
          """Return the MD5 hex digest of the whole file ? the canonical video_id.

          Streams the file in fixed-size chunks so multi-gigabyte recordings never load
          into memory at once.
          """
          hasher = hashlib.md5()
          with open(filepath, "rb") as f:
              for chunk in iter(lambda: f.read(_CHUNK_SIZE), b""):
                  hasher.update(chunk)
          return hasher.hexdigest()

```

11. assistant (2026-06-18T01:28:03.446Z)

Let me read the rest of the files in my area.

12. user (2026-06-18T01:28:04.885Z)

```
      """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).

      Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
      - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
        for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
        so it is abandoned, not joined ? it never blocks the process.
      - ``poll_until`` ? a bounded poll loop for an in-progress external op
```

```
(``poll_every_n_sec`` cadence,
        ``max_wait_sec`` deadline).

    **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-tested with zero real
    waiting (a fake clock advances instantly).

    **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at **DEBUG**; state
    transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean; enable DEBUG on the
    ``execution.policy`` logger to watch the poll trail.
    """
    from __future__ import annotations

    import logging
    import threading
    import time as _time
    from dataclasses import asdict, dataclass
    from enum import Enum
    from typing import Callable, Optional, TypeVar

    T = TypeVar("T")

    #: Dedicated logger so callers can dial polling visibility independently of the app's root level.
    LOG = logging.getLogger("execution.policy")


    class WaitMode(str, Enum):
        """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on the job row."""
        WAIT_AND_RESUME = "wait_and_resume"   # default: bounded wait; over budget -> reschedule/partial (P2/P3)
        ABORT = "abort"                       # one attempt; on hang/fail, stop immediately
        BLOCK = "block"                       # bounded in-process blocking wait (no reschedule)


    class PolicyTimeout(TimeoutError):
        """An op hung past ``op_timeout_sec`` or never became ready within ``max_wait_sec``."""


    @dataclass(frozen=True)
    class ExecutionPolicy:
        """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see EXECUTION_POLICY.md)."""
        mode: WaitMode = WaitMode.WAIT_AND_RESUME
        poll_every_n_sec: float = 10.0
        op_timeout_sec: float = 120.0         # HARD per-attempt deadline ? the hang-killer that was missing
        max_wait_sec: float = 1800.0          # total wall budget for a poll loop
        max_retries: int = 5                  # retries (=> max_retries+1 attempts) for failures/hangs
        backoff_base_sec: float = 3.0         # exponential: backoff_base * 2**attempt
        on_timeout: str = "reschedule"        # reschedule | partial | fail  (consumed by P2/P3)
        resumable: bool = True

        def to_dict(self) -> dict:
            d = asdict(self)
            d["mode"] = self.mode.value
            return d

```

```
        @classmethod
        def from_dict(cls, d: dict) -> "ExecutionPolicy":
            known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
            kw = {k: v for k, v in d.items() if k in known}
            if "mode" in kw and not isinstance(kw["mode"], WaitMode):
                kw["mode"] = WaitMode(kw["mode"])
            return cls(**kw)


    #: The sane default (also what a job gets if it declares no policy).
    DEFAULT_POLICY = ExecutionPolicy()


    def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
        """Deterministic exponential backoff (no jitter ? reproducible tests)."""
        return policy.backoff_base_sec * (2 ** attempt)


    def run_bounded(fn: Callable[[], T], policy: ExecutionPolicy = DEFAULT_POLICY, *,
                    label: str = "op", logger: Optional[logging.Logger] = None,
                    sleep: Callable[[float], None] = _time.sleep) -> T:
        """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with
backoff.

        Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt
hung, or re-raises the
        last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no
retries).
        """
        log = logger or LOG
        attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
        last_exc: Optional[BaseException] = None
        for attempt in range(attempts):
            box: dict = {}
            done = threading.Event()

            # box/done bound as defaults (not closed over) ? the thread is started + joined
within this
            # same iteration, but explicit binding keeps each attempt isolated and silences
B023.
            def _runner(_box: dict = box, _done: threading.Event = done) -> None:
                try:
                    _box["val"] = fn()
                except BaseException as e:        # relay ANY error (incl. timeouts) to the
caller thread
                    _box["exc"] = e
                finally:
                    _done.set()

            threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
            if done.wait(timeout=policy.op_timeout_sec):
                if "exc" in box:
                    last_exc = box["exc"]
                    log.info("%s: attempt %d/%d failed: %s", label, attempt + 1, attempts,
last_exc)
                else:
                    if attempt:
                        log.info("%s: succeeded on attempt %d/%d", label, attempt + 1,
attempts)
                    return box["val"]               # type: ignore[return-value]
            else:
                last_exc = PolicyTimeout(f"{label} hung >
op_timeout_sec={policy.op_timeout_sec}s")
                log.warning("%s: attempt %d/%d HUNG (>%.0fs) ? abandoning", label, attempt +
1, attempts,
```

```
115                             policy.op_timeout_sec)
116             if attempt < attempts - 1:
117                 wait = _backoff_sec(policy, attempt)
118                 log.debug("%s: backing off %.1fs before retry", label, wait)
119                 sleep(wait)
120         assert last_exc is not None
121         log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
122         raise last_exc
123
124
125     def poll_until(check: Callable[[], Optional[T]], policy: ExecutionPolicy =
DEFAULT_POLICY, *,
126                    label: str = "op", logger: Optional[logging.Logger] = None,
127                    sleep: Callable[[float], None] = _time.sleep,
128                    monotonic: Callable[[], float] = _time.monotonic) -> T:
129         """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
130         ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
131
132         Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
133         """
134         log = logger or LOG
135         start = monotonic()
136         polls = 0
137         log.info("%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
138                  label, policy.poll_every_n_sec, policy.max_wait_sec)
139         while True:
140             result = check()
141             if result is not None:
142                 log.info("%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() -
start)
143                 return result
144             elapsed = monotonic() - start
145             if elapsed >= policy.max_wait_sec:
146                 log.warning("%s: still not ready after %.0fs (max_wait) ? giving up", label,
elapsed)
147                 raise PolicyTimeout(f"{label} not ready within
max_wait_sec={policy.max_wait_sec}s")
148             polls += 1
149             log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
150             sleep(policy.poll_every_n_sec)
151
```

13. user (2026-06-18T01:28:05.346Z)

```
1       """Structured run telemetry ? a compact, rotating JSONL "black box" for long/flaky runs.
2
3       A long GPU/CV run (the fusion-golden extraction, a WSL/WASB pass, an ffmpeg stitch) can
die with
4       **no Python traceback** ? the OS kills it (a console-control event, an OOM, a thermal
event). When
5       that happens the only forensic evidence is whatever was last flushed to disk. This
module keeps a
6       tiny (~10 KB) rotating runlog of *execution velocity + machine health* so a post-mortem
is a
7       bullseye instead of a guess: "it was at 26 fps, GPU 85 °C throttled to 1125/2100 MHz,
4.8 GB VRAM
8       free, CPU 60 %, RAM 21/32 GB ? then vanished."
9
10      It is the **observability** companion to the `ExecutionPolicy` *control* layer
11      (`docs/EXECUTION_POLICY.md`): the policy decides wait / abort / resume; this records the
signals
12      that explain *why* a run slowed or died, and lets us compare execution velocity across
```

```
quality
13      iterations.
14
15      Design:
16      - **Caller-driven snapshots** (no background thread): call ``snapshot(done, total)``
from an
17        existing progress callback; fps is derived from the delta since the previous snapshot.
18        ``event(kind, msg)`` records discrete moments (start, video-done, error) WITH a health
sample, so
19        the last line before a kill captures the machine state at death.
20      - **Capped + rotating**: a fixed ``meta`` header line (machine / GPU / start) is always
kept; recent
21        snapshot/event lines form an on-disk ring trimmed to ``max_bytes`` (~10 KB). The file
is rewritten
22        atomically (tmp + ``os.replace``) on each write, so a hard kill never leaves it half-
written.
23      - **Graceful degradation**: GPU stats come from ``nvidia-smi`` (absent -> ``None``);
system stats
24        from ``psutil`` (not installed -> ``None``). Collection never raises into the caller ?
a telemetry
25        hiccup must not break the run it observes.
26      - **Deterministic / testable**: the clock and the two collectors are injectable, so fps,
rotation,
27        and parsing are unit-tested with no real GPU, no psutil, and no wall-clock.
28
29      All stdlib + optional ``psutil`` (BSD-3, commercial-clean). No network, no training-data
path.
30      """
31      from __future__ import annotations
32
33      import json
34      import os
35      import shutil
36      import subprocess
37      import time
38      from typing import Any, Callable, Dict, List, Optional
39
40      GpuFn = Callable[[], Optional[Dict[str, Any]]]
41      SysFn = Callable[[], Optional[Dict[str, Any]]]
42
43      # nvidia-smi fields pulled in one query, in this order (see gpu_stats parsing).
44      _GPU_QUERY =
"temperature.gpu,utilization.gpu,power.draw,clocks.sm,clocks.max.sm,memory.used,memory.free"
45      _THROTTLE_CLOCK_FRAC = 0.8   # SM clock below this fraction of max => flagged throttled
46      _THROTTLE_TEMP_C = 84.0      # ... or temperature at/above this (laptop Max-Q throttle
territory)
47
48
49      def _num(s: str) -> Optional[float]:
50          try:
51              return float(s)
52          except (TypeError, ValueError):
53              return None
54
55
56      def gpu_stats(timeout: float = 4.0) -> Optional[Dict[str, Any]]:
57          """One ``nvidia-smi`` sample as a dict, or ``None`` if nvidia-smi is unavailable /
errors.
58
59          Returns util %, temperature, power (W), current + max SM clock (MHz), VRAM used /
free (MB), and a
60          derived ``throttled`` flag (SM clock well below max, or hot). Pure read; never
raises.
61          """
62          exe = shutil.which("nvidia-smi")
```

```
        if not exe:
            return None
        try:
            out = subprocess.run(
                [exe, f"--query-gpu={_GPU_QUERY}", "--format=csv,noheader,nounits"],
                capture_output=True, text=True, timeout=timeout, check=True,
            ).stdout.strip()
        except (subprocess.SubprocessError, OSError):
            return None
        row = out.splitlines()[0] if out else ""
        parts = [p.strip() for p in row.split(",")]
        if len(parts) < 7:
            return None
        vals = [_num(p) for p in parts[:7]]
        temp, util, power, sm, sm_max, used, free = vals
        throttled = bool(temp is not None and temp >= _THROTTLE_TEMP_C)
        if sm is not None and sm_max:
            throttled = throttled or sm < _THROTTLE_CLOCK_FRAC * sm_max
        return {
            "util_pct": util, "temp_c": temp, "power_w": power,
            "sm_mhz": sm, "sm_max_mhz": sm_max,
            "vram_used_mb": used, "vram_free_mb": free,
            "throttled": throttled,
        }


    def system_stats() -> Optional[Dict[str, Any]]:
        """CPU % + RAM via ``psutil``, or ``None`` if psutil isn't installed. Never
raises."""
        try:
            import psutil  # optional dep (BSD-3); only used when present
        except ImportError:
            return None
        try:
            vm = psutil.virtual_memory()
            return {
                "cpu_pct": psutil.cpu_percent(interval=None),
                "ram_used_gb": round(vm.used / 1e9, 2),
                "ram_total_gb": round(vm.total / 1e9, 2),
                "ram_pct": vm.percent,
            }
        except Exception:  # noqa: BLE001 - psutil edge cases must not break the run
            return None


    def gpu_name() -> Optional[str]:
        """The GPU model string (for the runlog header), or ``None`` without nvidia-smi."""
        exe = shutil.which("nvidia-smi")
        if not exe:
            return None
        try:
            out = subprocess.run([exe, "--query-gpu=name", "--format=csv,noheader"],
                                 capture_output=True, text=True, timeout=4.0,
check=True).stdout
        except (subprocess.SubprocessError, OSError):
            return None
        return out.splitlines()[0].strip() if out else None


    class RunTelemetry:
        """A small rotating JSONL runlog of execution velocity + machine health.

        Call ``snapshot(done, total, phase=...)`` from a progress callback and ``event(kind,
msg)`` at
        milestones. The file holds a ``meta`` header line plus the most recent snapshot /
```

```
event lines
125         that fit within ``max_bytes`` (~10 KB), rewritten atomically on every write.
126         """
127
128         def __init__(self, path: str, label: str = "run", max_bytes: int = 10_000,
129                      gpu_fn: GpuFn = gpu_stats, sys_fn: SysFn = system_stats,
130                      clock: Callable[[], float] = time.time) -> None:
131             self.path = path
132             self.label = label
133             self.max_bytes = max_bytes
134             self._gpu_fn = gpu_fn
135             self._sys_fn = sys_fn
136             self._clock = clock
137             self._t0 = clock()
138             self._last_done: Optional[int] = None
139             self._last_t: Optional[float] = None
140             self._lines: List[str] = []   # recent body lines (on-disk ring, trimmed to
max_bytes)
141             self._meta = json.dumps({"meta": {
142                 "label": label, "started": self._iso(self._t0), "pid": os.getpid(),
143                 "gpu_name": gpu_name(), "cpu_count": os.cpu_count(),
144             }}, separators=(",", ":"))
145             self._write()
146
147         # -- public API ---------------------------------------------------------
148         def snapshot(self, done: Optional[int] = None, total: Optional[int] = None,
149                      phase: Optional[str] = None, **extra: Any) -> Dict[str, Any]:
150             """Record a velocity + health sample; ``fps`` is derived from the previous
snapshot."""
151             now = self._clock()
152             rec: Dict[str, Any] = {"t": self._iso(now), "elapsed_s": round(now - self._t0,
1)}
153             if phase is not None:
154                 rec["phase"] = phase
155             if done is not None:
156                 rec["done"] = done
157                 if total:
158                     rec["total"] = total
159                     rec["pct"] = round(100.0 * done / total, 1)
160                 if self._last_done is not None and self._last_t is not None and now >
self._last_t:
161                     rec["fps"] = round((done - self._last_done) / (now - self._last_t), 1)
162                 self._last_done, self._last_t = done, now
163             rec.update(extra)
164             self._attach_health(rec)
165             self._append(rec)
166             return rec
167
168         def event(self, kind: str, msg: str = "", **extra: Any) -> Dict[str, Any]:
169             """Record a milestone (start, video_done, error, restart) WITH a health
sample."""
170             now = self._clock()
171             rec: Dict[str, Any] = {"t": self._iso(now), "elapsed_s": round(now - self._t0,
1),
172                                    "event": kind}
173             if msg:
174                 rec["msg"] = msg
175             rec.update(extra)
176             self._attach_health(rec)
177             self._append(rec)
178             return rec
179
180         # -- internals ----------------------------------------------------------
181         def _attach_health(self, rec: Dict[str, Any]) -> None:
182             gpu = self._safe(self._gpu_fn)
```

```
            sys_ = self._safe(self._sys_fn)
            if gpu is not None:
                rec["gpu"] = gpu
            if sys_ is not None:
                rec["sys"] = sys_

        def _append(self, rec: Dict[str, Any]) -> None:
            self._lines.append(json.dumps(rec, separators=(",", ":")))
            budget = self.max_bytes - len(self._meta) - 1
            while len(self._lines) > 1 and sum(len(s) + 1 for s in self._lines) > budget:
                self._lines.pop(0)   # drop oldest body line; the meta header is always kept
            self._write()

        def _write(self) -> None:
            try:
                tmp = self.path + ".tmp"
                with open(tmp, "w", encoding="utf-8") as f:
                    f.write(self._meta + "\n")
                    for s in self._lines:
                        f.write(s + "\n")
                os.replace(tmp, self.path)
            except OSError:
                pass   # telemetry must never break the run it observes

        @staticmethod
        def _safe(fn: GpuFn) -> Optional[Dict[str, Any]]:
            try:
                return fn()
            except Exception:  # noqa: BLE001 - a collector hiccup must not break the run
                return None

        @staticmethod
        def _iso(t: float) -> str:
            return time.strftime("%Y-%m-%dT%H:%M:%S", time.localtime(t))

```

14. user (2026-06-18T01:28:05.854Z)

```
    import subprocess
    import logging
    from typing import Optional

    logger = logging.getLogger(__name__)

    def get_video_duration(video_path: str) -> float:
        """Returns the exact duration of a video file in seconds using ffprobe.
        Returns 0.0 on failure."""
        try:
            duration_output = subprocess.check_output([
                "ffprobe", "-v", "error", "-show_entries", "format=duration",
                "-of", "default=noprint_wrappers=1:nokey=1", video_path
            ]).decode().strip()
            return float(duration_output)
        except Exception as e:
            logger.error(f"Could not detect video duration for {video_path}: {e}")
            return 0.0

    def extract_video_chunk(video_path: str, start_time: float, duration: float,
                            output_path: str, reencode: bool = False,
                            scale_width: Optional[int] = None) -> bool:
        """Extracts a chunk from a video file.

        Args:
            reencode: If True, re-encodes with libx264 (slower but frame-accurate).
                      If False, uses stream-copy (fast but cuts at nearest keyframe).
```

```
                        Use True for short clips where keyframe drift matters.
            scale_width: If set (requires reencode), downscale to this width (height auto,
                        even). Useful for high-res sources headed to an API that analyses at
                        low res anyway ? shrinks uploads with no analysis loss.
        """
        cmd = [
            "ffmpeg", "-y",
            "-ss", str(start_time),
            "-t", str(duration),
            "-i", video_path,
        ]
        if reencode:
            if scale_width:
                cmd += ["-vf", f"scale={int(scale_width)}:-2"]
            cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
        else:
            cmd += ["-c", "copy"]
        cmd.append(output_path)
        try:
            subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
            return True
        except Exception as e:
            logger.error(f"Failed to extract chunk {start_time}s - {start_time+duration}s:
{e}")
            return False

```

15. assistant (2026-06-18T01:28:09.635Z)

Let me read the remaining files: config models/loader, the other utils, and features.

16. user (2026-06-18T01:28:10.588Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
29          max_scene_cuts_per_minute: float = 0.5
30          relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
```

```
31
32
33      class TrackNetConfig(BaseModel):
34          wsl_distro: str = "Ubuntu"
35          repo_dir: str = "~/models/TrackNetV4"
36          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37          conda_env: str = "TrackNetV4"
38          weights_path: str = ""
39          queue_length: int = 5
40          stage_in_wsl: bool = True
41          wsl_stage_dir: str = "~/clips"
42          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43          velocity_threshold: float = 0.02
44          # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45          # decoded frame cache) are deleted automatically ? they are pure intermediates,
46          # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47          # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48          keep_frames: bool = False
49          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50          # 0 = disabled.
51          min_free_gb: float = 0.0
52
53
54      class WasbIndexConfig(BaseModel):
55          """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57          Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58          these knobs actually take effect ? previously this whole block was silently dropped
59          because nested config sections defaulted to extra='ignore'.
60          """
61          wsl_distro: str = "Ubuntu"
62          repo_dir: str = "~/models/WASB-SBDT"
63          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64          conda_env: str = "wasb"
65          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66          sport: str = "badminton"
67          wsl_stage_dir: str = "~/clips"
68          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69          velocity_threshold: float = 0.02
70          # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
71          # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
72          # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
73          keep_frames: bool = False
74          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75          min_free_gb: float = 0.0
76          # FAST PATH (default OFF until owner-verified side-by-side): decode the staged video
77          # on the fly and feed frames straight to the detector, skipping the slow per-frame
PNG
78          # extraction that dominates a long clip (~46k PNGs for a 12-min 1080p60 video, which
79          # overran the 30-min timeout). Output is bit-identical to the PNG path (PNG is
lossless),
80          # so this is purely a speed/disk win; kept opt-in until the side-by-side confirms
it.
81          stream_video: bool = False
82
83
84      class FusionConfig(BaseModel):
85          """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
86
87          Every default reproduces control behaviour: the fusion segmenter persists the
```

```
88          shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
89          true ? the person-cue features, but it does NOT gate the served handoff unless a
90          threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
91          is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
92          supplied ? off-court persons (spectators / adjacent courts) are excluded.
93          """
94          # Which trajectory substrate seeds the windows (shuttle stays primary).
95          substrate: Literal["wasb", "tracknet"] = "wasb"
96          # Windowing velocity threshold for the fusion family (the engine reads
97          # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
98          # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
99          velocity_threshold: float = 0.02
100         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
101         # enabled ? so the default path never imports torch / touches the GPU.
102         cue_flags: dict[str, bool] = Field(default_factory=dict)
103         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
104         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
105         min_crossings: int = Field(default=0, ge=0)
106         # Served Gate B: when the person cue is on, drop windows with median in-court
players
107         # < this. 0 = OFF.
108         min_players: int = Field(default=0, ge=0)
109         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
110         court_polygon: Optional[list[list[float]]] = None
111         # Net line for the person two-sided-motion split (person features only ? the shuttle
112         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
113         net_axis: Literal["x", "y"] = "y"
114         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
115
116
117     class IndexingConfig(BaseModel):
118         default_segmenter: str = "yolo_hybrid"
119         use_gpu: bool = True
120         motion_threshold: float = 0.08
121         min_segment_duration: float = 3.0
122         dead_time_merge_gap: float = 2.0
123         chunk_duration: float = 90.0
124         chunk_overlap: float = 10.0
125         detector_model: str = "fasterrcnn_resnet50_fpn"
126         detector_score_threshold: float = 0.5
127         yolo_velocity_threshold: float = 0.15
128         yolo_frame_skip: int = 3
129         yolo_tracker: str = "bytetrack.yaml"
130         yolo_prefetch_workers: int = 2
131         min_action_window_duration: float = 2.0
132         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF
(default ?
133         # behaviour unchanged). When > 0, a candidate window longer than this many seconds
is
134         # recursively split at its largest internal inactivity gap (?
``window_min_split_gap`` s ?
135         # the most likely rally boundary), so one window can't swallow several rallies + the
dead
136         # time between them. Never merges, only cuts (recall can't drop). Measured
recommended value
137         # 12.0 (golden n=6: ?F1 +0.139, 6/0 sign-test, merge_rate 0.523?0.038); kept OFF
until promoted.
138         max_window_duration: float = 0.0
139         window_min_split_gap: float = 0.5  # min internal gap (s) that qualifies as a split
point
```

```
140         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
141         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
142         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
143         # Only cuts (recall can't drop). OFF by default until measured/promoted.
144         window_hard_cap: bool = False
145         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
146         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
147         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
148         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts. OFF by
149         # default. Measured win at inactivity_end=1.5/rally_thresh=0.20/min_density=0.30
(n=11: ?tolF1
150         # +0.047, sign 8/0 p=0.008, |?end| 5.7?3.8s) ? see docs/RALLY_QUALITY_RESEARCH.md.
151         window_inactivity_end: float = 0.0
152         inactivity_rally_thresh: float = 0.08
153         inactivity_min_density: float = 0.34
154         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
155         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
156         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
157         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
158         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
159         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
160         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
161         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
162         window_blob_fallback: bool = False
163         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
164         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
165         # Default 4.0 is do-no-harm on the n=11 golden corpus (rescues only the 2
continuously-active
166         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
167         window_blob_factor: float = 4.0
168         log_yolo_candidates: bool = True
169         store_yolo_candidates: bool = True
170         ai_handoff_padding: float = 2.0
171         ai_max_workers: int = 5
172         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
173         # chunks of a high-bitrate (4K) source blow past the provider upload cap
174         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
175         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
176         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
177         # Matches gemini_refine's --clip-scale-width default.
178         ai_chunk_scale_width: int = 1280
179         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
180         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
181         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
182         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
```

```
        # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
        skip_ai_handoff: bool = False
        # Optional debug knob: trim every video to the first N seconds before processing.
        debug_duration: Optional[float] = None

        tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
        wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
        fusion: FusionConfig = Field(default_factory=FusionConfig)

        @field_validator("default_segmenter")
        @classmethod
        def segmenter_must_be_registered(cls, v: str) -> str:
            # Registry check happens at runtime in main/segmenter selection.
            # Here we just allow any string for forward compatibility.
            return v

        @model_validator(mode="after")
        def _chunk_step_must_be_positive(self) -> "IndexingConfig":
            # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
            # step makes `while t < duration: t += step` loop forever, growing memory before
any
            # GPU work. Reject the bad config at load time instead.
            if self.chunk_overlap >= self.chunk_duration:
                raise ValueError(
                    f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
                    f"({self.chunk_duration}); otherwise chunking never advances."
                )
            return self


    class OutputConfig(BaseModel):
        default_highlights_name: str = "highlights.mp4"
        db_name: str = "sports_indexer.db"
        # Directory where the DB, highlight reels, and processed clips are written.
        # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
        output_dir: str = "output"


    class WatermarkConfig(BaseModel):
        """Branding overlay burned into each highlight clip during the per-clip re-encode
        (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
        (under `stitching.watermark` in config.json) to turn it off. The text is fully
        configurable. The concat stage stays stream-copy (-c copy)."""
        enabled: bool = True
        text: str = "Generated by Khelsutra.guru"
        logo_path: str = ""        # optional transparent PNG overlay; "" = no logo
        font_path: str = ""        # optional .ttf; "" = use the fontconfig `font` family
below
        font: str = "Sans"         # fontconfig family used when font_path is empty
        font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5)  # text height as a
fraction of frame height
        opacity: float = Field(default=0.85, ge=0.0, le=1.0)
        box: bool = True           # faint backing box behind the text for legibility
        margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
        position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"


    class StitchingConfig(BaseModel):
        begin_padding: float = 0.5
        end_padding: float = 0.5
        use_cache: bool = False
```

```
        min_shot_count: int = 1
        watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)


    class LoggingConfig(BaseModel):
        """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
        # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
        # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
        # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
        # them to WARNING; set true to restore full chatter for request-flow debugging.
        verbose_http: bool = False


    class SecurityConfig(BaseModel):
        # When set, /api/process video_path must resolve under this directory (hosted/tenant
        # mode). Default None preserves the single-user local 'any local file' workflow.
        allowed_video_root: Optional[str] = None

        # --- Chunked upload (B2, PR-2) ---
        # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
        # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
        # contain upload_dir or /api/process will reject the uploaded paths.
        upload_dir: str = "output/uploads"
        # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
        # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
        max_upload_mb: int = 4096
        max_part_mb: int = 95
        allowed_upload_extensions: List[str] = ["mp4", "mov"]


    class AppConfig(BaseModel):
        """Root configuration object.

        All runtime configuration should be accessed via an instance of this class
        (or the loader) rather than raw dict lookups.
        """

        sport: Sport = "badminton"
        ai_provider: str = "gemini"
        # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
        # the alias never deprecates underneath us. Eval reports record the run date
alongside.
        ai_model: str = "gemini-flash-latest"

        validation: ValidationConfig = Field(default_factory=ValidationConfig)
        indexing: IndexingConfig = Field(default_factory=IndexingConfig)
        output: OutputConfig = Field(default_factory=OutputConfig)
        stitching: StitchingConfig = Field(default_factory=StitchingConfig)
        security: SecurityConfig = Field(default_factory=SecurityConfig)
        logging: LoggingConfig = Field(default_factory=LoggingConfig)

        # Allow extra keys during transition so existing config.json files do not break.
        model_config = {
            "extra": "allow",
            "validate_assignment": True,
        }

        # Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
        # Will be removed once all call sites (validators, segmenters, etc.) move to typed
        # attribute access / the typed config. A nested *section* is returned as a plain
dict
        # via model_dump, so legacy chained access keeps working ? returning the Pydantic
        # section model instead breaks it (a model has no .get), which bit every validator
```

```
on
302         # real runs. Scalar leaves are returned as-is.
303         def get(self, key: str, default: Any = None) -> Any:
304             _MISSING = object()
305             val: Any = _MISSING
306             if key in type(self).model_fields or hasattr(self, key):
307                 val = getattr(self, key, _MISSING)
308             if val is _MISSING:
309                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
310                 # discovered dynamically from the model fields, so a future top-level
section
311                 # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
312                 for fname in type(self).model_fields:
313                     section = getattr(self, fname, None)
314                     if isinstance(section, BaseModel) and hasattr(section, key):
315                         val = getattr(section, key)
316                         break
317             if val is _MISSING:
318                 return default
319             if isinstance(val, BaseModel):
320                 return val.model_dump(mode="json")
321             return val
322
323         def __getitem__(self, key: str) -> Any:
324             val = self.get(key)
325             if val is None and key not in type(self).model_fields:
326                 # Re-raise like a real dict for unknown top-level
327                 raise KeyError(key)
328             return val
329
330
331     # Convenience type alias for callers that want to annotate
332     Config = AppConfig
333
```

17. user (2026-06-18T01:28:11.003Z)

```
1       """Configuration loader and saver.
2
3       Provides a single place to load validated AppConfig with built-in defaults
4       merged with optional file overrides (config.json).
5
6       This replaces scattered json.load + .get() patterns throughout the codebase.
7       """
8
9       from __future__ import annotations
10
11      import json
12      import logging
13      from pathlib import Path
14      from typing import Any
15
16      from pydantic import BaseModel
17
18      from .models import AppConfig
19
20      logger = logging.getLogger(__name__)
21
22      DEFAULT_CONFIG_PATH = Path("config.json")
23
24
25      def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
"") -> list[str]:
```

```
        """Dotted paths in `data` that the typed config will not honor.

        Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
        kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
        typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
        user's intent is lost ? collect every such key so load_config can warn.
        """
        unknown: list[str] = []
        fields = model_cls.model_fields
        for key, val in data.items():
            if key not in fields:
                unknown.append(prefix + key)
                continue
            ann = fields[key].annotation
            if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
                unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
        return unknown


    def _get_builtin_defaults() -> dict[str, Any]:
        """Return a plain dict of the current built-in defaults.

        The AppConfig model is the single source of truth for defaults.
        """
        return AppConfig().model_dump(mode="json")


    def load_config(path: str | Path | None = None) -> AppConfig:
        """Load configuration with the following precedence:

        1. Built-in defaults defined in the Pydantic models.
        2. Values from the JSON file (if present and readable).
        3. (Future) environment / CLI overrides.

        Missing file or unreadable file ? returns a fully defaulted AppConfig.
        Unknown keys in the file are tolerated (extra="allow" on the model).
        """
        cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH

        file_data: dict[str, Any] = {}
        if cfg_path.exists():
            try:
                with open(cfg_path, "r", encoding="utf-8") as f:
                    file_data = json.load(f)
            except Exception as exc:
                # Non-fatal: continue with defaults + whatever we could parse.
                # In production we might log here.
                logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")

        if file_data:
            unknown = _unknown_key_paths(file_data, AppConfig)
            if unknown:
                logger.warning(
                    "[CONFIG WARNING] %s contains keys the typed config does not "
                    "recognize (typo'd or removed knobs ? top-level extras are kept "
                    "but unread; unknown section keys are silently dropped): %s",
                    cfg_path, ", ".join(sorted(unknown)),
                )

        # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
        merged = {**_get_builtin_defaults(), **file_data}

        return AppConfig.model_validate(merged)
```

```
89
90
91      def save_default_config(path: str | Path | None = None) -> Path:
92          """Write a fresh default config.json.
93
94          Used by scripts/doctor.py diagnostics and the --setup CLI path.
95          Overwrites if the file exists.
96          """
97          cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
98          cfg = AppConfig()
99          cfg_path.parent.mkdir(parents=True, exist_ok=True)
100         cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
101         return cfg_path
102
103
104     # Backwards-compatible alias used by some older call sites during transition.
105     def get_default_config() -> dict[str, Any]:
106         """Return defaults as a plain dict (for transitional use only)."""
107         return _get_builtin_defaults()
108
```

18. assistant (2026-06-18T01:28:13.445Z)

Now let me read the remaining utils files and the features file.

19. user (2026-06-18T01:28:14.475Z)

```
1       import math
2       from typing import Tuple
3
4       def get_center(bbox: Tuple[float, float, float, float]) -> Tuple[float, float]:
5           """Calculate the center (x, y) of a bounding box.
6           bbox format: (x1, y1, x2, y2)
7           """
8           x1, y1, x2, y2 = bbox
9           return ((x1 + x2) / 2.0, (y1 + y2) / 2.0)
10
11      def get_distance(p1: Tuple[float, float], p2: Tuple[float, float]) -> float:
12          """Calculate the Euclidean distance between two points (x, y)."""
13          return math.sqrt((p2[0] - p1[0])**2 + (p2[1] - p1[1])**2)
14
15      def get_velocity(bbox1: Tuple[float, float, float, float], bbox2: Tuple[float, float,
float, float], fps: float) -> float:
16          """Calculate velocity (pixels per second) of a bounding box moving between two
consecutive frames.
17          """
18          c1 = get_center(bbox1)
19          c2 = get_center(bbox2)
20          dist = get_distance(c1, c2)
21          return dist * fps
22
23      def get_velocity_normalised(bbox1: Tuple[float, float, float, float], bbox2:
Tuple[float, float, float, float],
24                                  fps: float, frame_width: float) -> float:
25          """Calculate velocity as a fraction of frame width per second.
26
27          This makes the threshold resolution-independent:
28          - 0.15 means the center moved 15% of the frame width in one second.
29          - Works identically on 720p, 1080p, and 4K footage.
30
31          Args:
32              bbox1: Previous bounding box (x1, y1, x2, y2).
33              bbox2: Current bounding box (x1, y1, x2, y2).
34              fps: Effective frames per second (after frame-skipping).
35              frame_width: Width of the video frame in pixels.
```

```
        """
        raw_velocity = get_velocity(bbox1, bbox2, fps)
        if frame_width <= 0:
            return raw_velocity  # Fallback to raw pixels if width unknown
        return raw_velocity / frame_width

    def calculate_iou(boxA: Tuple[float, float, float, float], boxB: Tuple[float, float,
float, float]) -> float:
        """Calculate the Intersection over Union (IoU) of two bounding boxes.
        bbox format: (x1, y1, x2, y2)
        """
        xA = max(boxA[0], boxB[0])
        yA = max(boxA[1], boxB[1])
        xB = min(boxA[2], boxB[2])
        yB = min(boxA[3], boxB[3])

        interArea = max(0, xB - xA + 1) * max(0, yB - yA + 1)
        if interArea == 0:
            return 0.0

        boxAArea = (boxA[2] - boxA[0] + 1) * (boxA[3] - boxA[1] + 1)
        boxBArea = (boxB[2] - boxB[0] + 1) * (boxB[3] - boxB[1] + 1)

        iou = interArea / float(boxAArea + boxBArea - interArea)
        return iou

```

20. user (2026-06-18T01:28:15.358Z)

```
    """Path-safety helpers for the API boundary (untrusted request fields)."""
    import os
    from typing import Optional


    def safe_output_filename(name: str) -> str:
        """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).

        `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
absolute,
        and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-
write
        primitive. Returns the name unchanged when safe; raises ValueError otherwise.
        """
        if not name or not name.strip():
            raise ValueError("output_filename must not be empty")
        if "\x00" in name:
            raise ValueError("output_filename must not contain a null byte")
        if "/" in name or "\\" in name:
            raise ValueError(f"output_filename must not contain a path separator: {name!r}")
        if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
            raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
        return name


    def resolve_under_root(path: str, root: str) -> str:
        """Return the real path, asserting it is contained under `root` (raises ValueError
if not)."""
        rp = os.path.realpath(path)
        rr = os.path.realpath(root)
        try:
            contained = os.path.commonpath([rp, rr]) == rr
        except ValueError:  # different drives on Windows
            contained = False
        if not contained:
```

```
33              raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
34          return rp
35
36
37      def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
38          """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
39
40          With no allowed_root (default, single-user local mode) only the null-byte check
applies,
41          so the local 'point it at any of my videos' workflow is preserved. Setting an
42          allowed_root (a future hosted/tenant deployment) enforces containment.
43          """
44          if "\x00" in path:
45              raise ValueError("video_path must not contain a null byte")
46          if allowed_root:
47              resolve_under_root(path, allowed_root)
48
```

21. user (2026-06-18T01:28:15.697Z)

```
1       """Per-frame rally features over a shuttle/ball trajectory ? the rally_seq feature
schema.
2
3       THE single source of truth for the Gen-0 TAL feature extraction (ADR-008): the training
4       harness (training/) and the serving path import THIS module, so train == serve by
5       construction. The fps-normalization bug fixed on 2026-06-10 (speed read ~2x smaller at
6       59.94fps than 30fps) is exactly the train/serve-skew class this placement prevents.
7
8       Pure numpy/scipy ? importable without torch/cv2. Any change to the feature semantics
9       MUST bump FEATURE_SCHEMA_VERSION; artifact manifests record the version they were
10      trained with and the serving loader refuses a mismatch.
11      """
12      from __future__ import annotations
13
14      from typing import Dict
15
16      import numpy as np
17      from scipy.ndimage import uniform_filter1d, maximum_filter1d
18
19      # Bump on ANY semantic change to build_frame_arrays/features (names, normalization,
20      # windowing, occlusion rule). Artifacts pin this; the loader hard-fails on mismatch.
21      FEATURE_SCHEMA_VERSION = 1
22
23      FEAT_NAMES = ["vis_rate", "spd_mean", "spd_max", "cross_rate", "x_std", "y_std"]
24
25      # Default windowing ? part of the schema (manifests record the values actually used).
26      DEFAULT_WIN_SEC = 0.75
27      DEFAULT_STRIDE_SEC = 0.1
28
29      FEATURE_SCHEMA = {
30          "id": "rally_seq",
31          "version": FEATURE_SCHEMA_VERSION,
32          "feat_names": FEAT_NAMES,
33          "win_sec": DEFAULT_WIN_SEC,
34          "stride_sec": DEFAULT_STRIDE_SEC,
35          # Explicit because its absence was a real bug: speed is fraction-of-frame-width
36          # PER SECOND (* fps), matching the heuristic windowing brain.
37          "fps_semantics": "per-second",
38          "occlusion_rule": "speed/crossings not computed across gaps > max(2,
round(0.25*fps)) frames",
39          "input": "trajectory points (frame, visible, x, y) + REQUIRED runtime metadata {fps,
frame_width}",
40      }
41
42
```

```
    def _spread(a: np.ndarray) -> float:
        return float(np.percentile(a, 95) - np.percentile(a, 5)) if a.size > 1 else 0.0


    def build_frame_arrays(points, frame_width: float, fps: float = 30.0) -> Dict:
        """Per-frame trajectory arrays + per-visible-frame speed and net-crossing events.

        Speed is normalized to fraction-of-frame-width PER SECOND (``* fps``), matching the
        heuristic windowing brain ? previously it was per-frame, so the same physical
shuttle
        speed read ~2x smaller at 59.94 fps than at 30 fps, confounding cross-footage
transfer
        on a corpus that mixes frame rates. Speed/crossings are also not computed across a
long
        occlusion gap (mirrors the heuristic resetting on invisibility)."""
        pts = sorted(points, key=lambda p: p.frame)
        N = (pts[-1].frame + 1) if pts else 1
        vis = np.zeros(N)
        x = np.zeros(N)
        y = np.zeros(N)
        for p in pts:
            if 0 <= p.frame < N and p.visible and p.x >= 0:
                vis[p.frame] = 1.0
                x[p.frame] = float(p.x)
                y[p.frame] = float(p.y)

        mask = vis > 0
        play_is_x = _spread(x[mask]) >= _spread(y[mask])
        coord = x if play_is_x else y
        net = float(np.median(coord[mask])) if mask.any() else 0.0
        deadband = 0.06 * (_spread(coord[mask]) if mask.sum() > 1 else 1.0)

        speed = np.zeros(N)
        spd_mask = np.zeros(N)
        cross = np.zeros(N)
        max_gap = max(2, round(0.25 * fps))  # frames; a longer gap = occlusion break, not
motion
        prev = None
        for p in pts:
            f = p.frame
            if not (0 <= f < N and vis[f]):
                continue
            if prev is not None:
                df = f - prev
                if 0 < df <= max_gap:
                    speed[f] = float(np.hypot(x[f] - x[prev], y[f] - y[prev]) / frame_width
/ df * fps)
                    spd_mask[f] = 1.0
                    c, pc = coord[f] - net, coord[prev] - net
                    if abs(c) > deadband and abs(pc) > deadband and (c > 0) != (pc > 0):
                        cross[f] = 1.0
            prev = f
        return dict(N=N, vis=vis, x=x, y=y, speed=speed, spd_mask=spd_mask, cross=cross,
fw=frame_width)


    def features(arr: Dict, fps: float, win_sec: float = DEFAULT_WIN_SEC,
                 stride_sec: float = DEFAULT_STRIDE_SEC):
        """Windowed per-frame features sampled at a stride -> (X[n_samples,6],
times[n_samples])."""
        N = arr["N"]
        fw = arr["fw"]
        size = max(3, int(win_sec * fps) * 2 + 1)
        vis, speed, spd_mask, cross, x, y = (arr[k] for k in ("vis", "speed", "spd_mask",
"cross", "x", "y"))
```

```
100
101         n = uniform_filter1d(vis, size, mode="constant")                        # mean
visibility (== vis_rate)
102         sm = uniform_filter1d(spd_mask, size, mode="constant")
103         spd_mean = uniform_filter1d(speed, size, mode="constant") / np.maximum(sm, 1e-9)
104         spd_max = maximum_filter1d(speed, size, mode="constant")
105         cross_rate = uniform_filter1d(cross, size, mode="constant") * fps       # crossings
/ second
106         xm = uniform_filter1d(x * vis, size, mode="constant") / np.maximum(n, 1e-9)
107         x2 = uniform_filter1d((x * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
108         x_std = np.sqrt(np.maximum(x2 - xm ** 2, 0.0)) / fw
109         ym = uniform_filter1d(y * vis, size, mode="constant") / np.maximum(n, 1e-9)
110         y2 = uniform_filter1d((y * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
111         y_std = np.sqrt(np.maximum(y2 - ym ** 2, 0.0)) / fw
112
113         idx = np.arange(0, N, max(1, int(stride_sec * fps)))
114         X = np.stack([n[idx], spd_mean[idx], spd_max[idx], cross_rate[idx], x_std[idx],
y_std[idx]], axis=1)
115         return np.nan_to_num(X), idx / fps
116
```

22. assistant (2026-06-18T01:28:23.572Z)

Now let me look closely at a few candidate areas. Let me examine the `claim_due_job` ordering logic and the `seconds_until_next_due` more carefully, plus the `_append` ring-buffer logic in run_telemetry.

Let me first investigate the `claim_due_job` ordering. The SELECT orders by `COALESCE(next_poll_at, created_at)`. For a `queued` job, `next_poll_at` is NULL, so it orders by `created_at`. For a `waiting` job, it orders by `next_poll_at`. These two timestamps are in different formats potentially. Let me check the telemetry ring buffer first since that has a concrete logic concern.

In `run_telemetry._append`:
```python
budget = self.max_bytes - len(self._meta) - 1
while len(self._lines) > 1 and sum(...) > budget:
```
The `while len(self._lines) > 1` keeps at least one body line. That's fine. Let me look at a potential issue with `compute_video_id` and check the geometry IoU `+1` convention. Let me check tests to verify contracts.

23. user (2026-06-18T01:28:25.528Z)

```
__init__.py
test_ablation.py
test_annotation_providers.py
test_api_endpoints.py
test_async_jobs.py
test_audio_features.py
test_automated_provider.py
test_base.py
test_calibrate_local.py
test_calibrate_wasb.py
test_calibration.py
test_cleanup_caches.py
test_compiler.py
test_config.py
test_config_wasb.py
test_database.py
test_detector_keying.py
test_distill_local.py
test_eval_annotations.py
test_eval_batch.py
test_eval_classifier.py
```

```
test_eval_harness.py
test_eval_metrics.py
test_eval_partial_labels.py
test_eval_regression.py
test_eval_stats.py
test_eval_training.py
test_execution_policy.py
test_experiment.py
test_experiment_honest.py
test_export_wasb_segments.py
test_fusion_compare.py
test_fusion_features.py
test_fusion_golden_telemetry.py
test_fusion_hybrid.py
test_gemini.py
test_gemini_refine.py
test_generate_highlights.py
test_geometry.py
test_golden_fixtures.py
test_golden_real_fixtures.py
test_gt_loader.py
test_hashing.py
test_hit_detector.py
test_import_smoke.py
test_job_self_scheduling.py
test_job_worker_loop.py
test_logging_setup.py
test_nightly_regression.py
test_path_safety.py
test_per_user_eval.py
test_per_user_gate.py
test_per_user_regression.py
test_person_detector.py
test_personalization_serving.py
test_promotion.py
test_provider_gemini.py
test_providers.py
test_rally_gate.py
test_rally_seg_eval.py
```

24. user (2026-06-18T01:28:26.061Z)

```
Found 17 files
docs\CODE_MAP.md
backend\pipeline\segmenters\fusion_hybrid.py
training\gen0\harness.py
backend\eval\fusion_golden.py
backend\eval\fusion_audio.py
backend\pipeline\detectors\person_detector.py
tests\test_audio_features.py
backend\pipeline\detectors\audio_features.py
tests\test_fusion_features.py
backend\pipeline\detectors\fusion_features.py
tests\test_rally_seq_proto.py
tests\test_geometry.py
backend\features\rally_seq.py
backend\eval\rally_seq_proto.py
tests\test_eval_training.py
backend\utils\geometry.py
backend\eval\training.py
```

25. assistant (2026-06-18T01:28:28.418Z)

```
Let me look at the `claim_due_job` ordering bug more carefully and at how `next_poll_at` vs
`created_at` compare. Let me check the test for self-scheduling and the database test.
```

26. user (2026-06-18T01:28:29.690Z)

```
1       """Tests for the v3 self-scheduling job substrate (EXECUTION_POLICY.md P2).
2
3       Covers the policy column round-trip, set_job_waiting, and the atomic claim_due_job
4       (queued + due-waiting claimable, not-yet-due skipped, drained-then-None, compare-and-
set).
5       """
6       from backend.infrastructure.database import Database
7
8       POLICY = {"mode": "wait_and_resume", "op_timeout_sec": 90.0, "max_retries": 3}
9
10
11      def _db(tmp_path):
12          return Database(str(tmp_path / "jobs.db"))
13
14
15      def test_policy_roundtrips_through_create_and_get(tmp_path):
16          db = _db(tmp_path)
17          assert db.create_job("j1", "/v.mp4", policy=POLICY)
18          assert db.get_job("j1")["policy"] == POLICY
19
20
21      def test_create_without_policy_is_none(tmp_path):
22          db = _db(tmp_path)
23          db.create_job("j0", "/v.mp4")
24          assert db.get_job("j0")["policy"] is None
25
26
27      def test_set_job_policy_replaces(tmp_path):
28          db = _db(tmp_path)
29          db.create_job("j1", "/v.mp4")
30          assert db.set_job_policy("j1", {"mode": "abort"})
31          assert db.get_job("j1")["policy"] == {"mode": "abort"}
32
33
34      def test_set_job_waiting_sets_status_and_due(tmp_path):
35          db = _db(tmp_path)
36          db.create_job("j1", "/v.mp4")
37          assert db.set_job_waiting("j1", delay_sec=3600, progress="waiting on gemini")
38          row = db.get_job("j1")
39          assert row["status"] == "waiting" and row["next_poll_at"] is not None
40          assert row["progress"] == "waiting on gemini"
41
42
43      def test_claim_due_job_none_when_empty(tmp_path):
44          assert _db(tmp_path).claim_due_job() is None
45
46
47      def test_claim_due_job_claims_queued_and_flips_running(tmp_path):
48          db = _db(tmp_path)
49          db.create_job("j1", "/v.mp4", policy=POLICY)
50          claimed = db.claim_due_job()
51          assert claimed["id"] == "j1" and claimed["status"] == "running"
52          assert claimed["policy"] == POLICY                      # policy travels with the
claim
53          assert db.get_job("j1")["status"] == "running"
54
55
56      def test_claim_drains_then_returns_none(tmp_path):
57          db = _db(tmp_path)
58          db.create_job("a", "/a.mp4")
59          db.create_job("b", "/b.mp4")
60          got = {db.claim_due_job()["id"], db.claim_due_job()["id"]}
61          assert got == {"a", "b"}                                # both drained (order not
```

```
asserted)
62          assert db.claim_due_job() is None                       # nothing runnable left
63
64
65      def test_waiting_job_due_is_claimable(tmp_path):
66          db = _db(tmp_path)
67          db.create_job("j1", "/v.mp4")
68          db.set_job_waiting("j1", delay_sec=0)                   # due immediately
69          claimed = db.claim_due_job()
70          assert claimed is not None and claimed["id"] == "j1" and claimed["status"] ==
"running"
71
72
73      def test_waiting_job_not_due_is_skipped(tmp_path):
74          db = _db(tmp_path)
75          db.create_job("j1", "/v.mp4")
76          db.set_job_waiting("j1", delay_sec=3600)                # due in an hour
77          assert db.claim_due_job() is None                       # not yet due -> not
claimed
78          assert db.get_job("j1")["status"] == "waiting"          # left parked
79
80
81      def test_running_job_is_not_reclaimed(tmp_path):
82          db = _db(tmp_path)
83          db.create_job("j1", "/v.mp4")
84          db.claim_due_job()                                      # -> running
85          assert db.claim_due_job() is None                       # a running job is not
runnable
86
```

27. assistant (2026-06-18T01:28:39.527Z)

Now let me focus on the `claim_due_job` ordering. The `ORDER BY COALESCE(next_poll_at, created_at)` compares timestamps. `created_at` defaults to `CURRENT_TIMESTAMP` (format `YYYY-MM-DD HH:MM:SS`) and `next_poll_at` is set via `datetime('now', ?)` which is the same format. So they're comparable. Not a bug.

Let me look more carefully at the telemetry `_append`. The budget calculation: when a single line is larger than budget, the loop stops at `len > 1`. Fine.

Let me look at the `gpu_stats` parsing edge. `vals = [_num(p) for p in parts[:7]]` then unpacks 7 values. `parts` has `>= 7` guaranteed. OK.

Let me examine the `features()` function in rally_seq for the `spd_max` with `mode="constant"`. `maximum_filter1d` with `mode="constant"` uses `cval=0.0` by default ? padding with 0 for max is fine since speed >= 0.

Let me look at the `idx` indexing in `features()`: `idx = np.arange(0, N, max(1, int(stride_sec*fps)))`. These index into arrays of length N. Safe.

Let me dig into a potential real bug: `build_frame_arrays` with `N = (pts[-1].frame + 1) if pts else 1`. If a point has a negative frame or frame beyond, the loop guards `0 <= p.frame < N`. But `pts[-1].frame` after sort is the max frame. If all frames are negative? Then N could be <= 0. Let me check. If `pts[-1].frame` is negative (say -5), N = -4, and `np.zeros(-4)` raises ValueError. Let me verify whether negative frames are possible. Let me check the data contract / Point type.

28. user (2026-06-18T01:28:41.146Z)

```
Found 18 files
tests\test_tracknet_runner.py
tests\test_golden_real_fixtures.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\detectors\tracknet_runner.py
```

backend\eval\golden_fixtures.py
backend\pipeline\detectors\wasb_infer.py
backend\eval\fusion_golden.py
backend\pipeline\detectors\person_detector.py
tests\test_rally_gate.py
backend\pipeline\detectors\fusion_features.py
backend\pipeline\detectors\rally_gate.py
backend\features\rally_seq.py
backend\eval\eval_partial_labels.py
backend\eval\calibrate_wasb.py
backend\eval\export_wasb_segments.py
backend\eval\audio\e1_probe.py
backend\eval\gemini_refine.py

29. user (2026-06-18T01:28:41.840Z)

```
1       """Smoke + sanity for the learned per-frame rally segmenter prototype.
2
3       Not a quality bar (it's a prototype) ? just pins that the feature/label/segment plumbing
4       works and that a clearly rally-vs-dead synthetic trajectory is separable (AUC well above
5       chance), so the proof-of-signal claim is reproducible.
6       """
7       import csv
8
9       import numpy as np
10      import pytest
11
12      from backend.eval import rally_seq_proto as rp
13
14
15      def _write_traj(path, fps=10.0):
16          """120s synthetic trajectory: shuttle oscillates fast across the net during
'rallies'
17          (0-20s, 40-60s, 80-100s) and is mostly still/invisible in the dead time between."""
18          rows = [("Frame", "Visibility", "X", "Y")]
19          n = int(120 * fps)
20          for f in range(n):
21              t = f / fps
22              in_rally = (t < 20) or (40 <= t < 60) or (80 <= t < 100)
23              if in_rally:
24                  x = 50 + 45 * np.sin(2 * np.pi * (f % 10) / 10.0)  # crosses net (x=50)
repeatedly
25                  rows.append((f, 1, round(float(x), 2), 50))
26              else:
27                  if f % 5 == 0:                                      # mostly invisible,
occasional still blip
28                      rows.append((f, 1, 50.0, 50))
29                  else:
30                      rows.append((f, 0, -1, -1))
31          with open(path, "w", newline="") as fh:
32              csv.writer(fh).writerows(rows)
33          return str(path)
34
35
36      def _write_labels(path):
37          with open(path, "w", newline="", encoding="utf-8") as fh:
38              w = csv.writer(fh)
39              w.writerow(["rally_number", "start_time", "end_time"])
40              w.writerow([1, 0.0, 20.0])
41              w.writerow([2, 40.0, 60.0])
42              w.writerow([3, 80.0, 100.0])
43          return str(path)
44
45
46      def test_features_and_labels_align(tmp_path):
```

```
47          from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
48          pts = TrackNetRunner.parse_trajectory_csv(_write_traj(tmp_path / "t.csv", fps=10.0))
49          arr = rp.build_frame_arrays(pts, frame_width=100.0, fps=10.0)
50          X, times = rp.features(arr, fps=10.0)
51          assert X.shape[1] == len(rp.FEAT_NAMES) and X.shape[0] == len(times)
52          assert np.isfinite(X).all()
53          y = rp.labels_for(times, [(0.0, 20.0), (40.0, 60.0), (80.0, 100.0)])
54          assert 0 < y.sum() < len(y)            # both classes present
55
56
57      def test_signal_is_separable(tmp_path):
58          """The whole point: rally vs dead-time is separable from the trajectory features."""
59          res = rp.run([{
60              "name": f"v{i}", "trajectory": _write_traj(tmp_path / f"t{i}.csv", fps=10.0),
61              "fps": 10.0, "frame_width": 100.0, "labels": _write_labels(tmp_path /
f"g{i}.csv"),
62          } for i in range(2)])
63          assert len(res["per_video"]) == 2
64          for r in res["per_video"]:
65              assert r["auc"] > 0.75             # well above chance (held-out)
66
67
68      def test_roc_auc_handles_ties():
69          # All probabilities tied -> AUC must be 0.5 (midranks), not an order-dependent
value.
70          y = np.array([0.0, 1.0, 0.0, 1.0])
71          p = np.full(4, 0.5)
72          assert rp.roc_auc(y, p) == pytest.approx(0.5)
73          # Reordering the same tied data must not change the result.
74          assert rp.roc_auc(y[::-1], p) == pytest.approx(0.5)
75          # A perfectly separating score still gives AUC 1.0.
76          assert rp.roc_auc(np.array([0, 0, 1, 1.0]), np.array([0.1, 0.2, 0.8, 0.9])) ==
pytest.approx(1.0)
77
78
79      def test_probs_to_segments_merges_and_filters():
80          times = np.arange(0, 10, 0.5)
81          probs = np.where((times >= 2) & (times <= 5), 0.9, 0.1)   # one clear block
82          segs = rp.probs_to_segments(probs, times, threshold=0.5, merge_gap=1.0, min_dur=1.0,
smooth=1)
83          assert len(segs) == 1
84          s, e = segs[0]
85          assert s <= 2.5 and e >= 4.5
86
```

30. assistant (2026-06-18T01:28:48.789Z)

Let me check the `parse_trajectory_csv` in tracknet_runner to understand the Point contract ? whether frame can be negative. Let me focus on the `_spread` and `net` computation when `mask` is all-zero. Let me look at a real concern in `build_frame_arrays`: when `mask.any()` is False (no visible points). Then `net = 0.0`, `deadband = 0.06 * 1.0`. The `coord` array is all zeros. That's defensively handled.

But there's a subtle concern: `play_is_x = _spread(x[mask]) >= _spread(y[mask])`. If `mask` is all-False, `x[mask]` is empty, `_spread` returns 0.0 (since `a.size > 1` is False). OK.

Let me now look at whether `pts[-1].frame` could be negative. Let me check `parse_trajectory_csv`.

31. user (2026-06-18T01:28:50.366Z)

```
237:    def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
```

32. user (2026-06-18T01:28:56.518Z)

```
                logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
                return None
            logger.info("TrackNet trajectory CSV: %s", expected_csv_win)

            # Success ? the CSV is the durable output; drop the staged video copy (a pure,
            # regenerable intermediate). On earlier failure we return above without
cleaning.
            if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
                logger.info("Cache hygiene: removing staged video for %s "
                            "(set indexing.tracknet.keep_frames=true to retain).", key)
                self._wsl_rm_rf([wsl_video])
            return expected_csv_win

        def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
            """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).

            Returns the WSL path to the staged copy, or None on failure.
            """
            src_mnt = to_wsl_mnt_path(video_win_path)
            reason = self.wsl_source_unreadable_reason(src_mnt)
            if reason:
                logger.error("Cannot stage video into WSL: %s", reason)
                return None
            dest = f"{self.cfg.wsl_stage_dir}/{base}"
            cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("Staging video into WSL timed out.")
                return None
            if res.returncode != 0 or "OK" not in (res.stdout or ""):
                logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
                return None
            return dest

        # ------------------------------------------------------------- CSV -> windows

        @staticmethod
        def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
            """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
            points: List[TrajectoryPoint] = []
            with open(csv_win_path, newline="") as f:
                reader = csv.DictReader(f)
                for row in reader:
                    try:
                        points.append(TrajectoryPoint(
                            frame=int(row["Frame"]),
                            visible=int(row["Visibility"]) == 1,
                            x=float(row["X"]),
                            y=float(row["Y"]),
                        ))
                    except (KeyError, ValueError):
                        continue
            points.sort(key=lambda p: p.frame)
            return points

        @staticmethod
        def trajectory_to_action_windows(
            points: List[TrajectoryPoint],
            fps: float,
            frame_width: float,
            chunk_start: float = 0.0,
```

```
261             velocity_thresh: float = 0.02,
262             merge_gap: float = 2.0,
263             min_window_duration: float = 2.0,
264             chunk_index: Optional[int] = None,
265             max_window_duration: float = 0.0,
266             min_split_gap: float = 0.5,
267             window_hard_cap: bool = False,
268             window_inactivity_end: float = 0.0,
269             inactivity_min_density: float = 0.34,
270             inactivity_rally_thresh: float = 0.08,
271             window_blob_fallback: bool = False,
272             window_blob_factor: float = 4.0,
273         ) -> List[Dict[str, Any]]:
274             """Convert a shuttle trajectory into candidate rally windows.
275
276             A frame is "active" when the shuttle is visible AND its inter-frame
277             displacement (normalised by frame width, per second) exceeds
278             ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
279             ``merge_gap`` seconds of each other are grouped into one window; short
280             windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
281             shape feeding the AI-handoff phase is identical.
282
283             ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
284             the *over-merge* failure where "shuttle moving" is conflated with "rally in
progress"
285             and a single window swallows several rallies + the dead time between them. When
set, any
286             window longer than this is recursively SPLIT at its largest internal inactivity
gap (the
287             longest stretch with no in-flight shuttle ? the most likely rally boundary),
provided
288             that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
fix from
289             ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
recall cannot
```

33. assistant (2026-06-18T01:29:12.242Z)

The CSV parse uses `int(row["Frame"])` which could in principle be negative, but in practice TrackNet frame indices start at 0. This is a theoretical edge not a reachable bug for the maintainer. I'll drop it.

Let me focus on more concrete things. Let me re-examine `upsert_user_model` carefully. There's a subtle issue with the active-demotion logic when `version` matches. Let me trace it:

When `is_active=True`, it demotes other active rows `WHERE ... is_active=1 AND version != ?`. Then it inserts/upserts the current row with `is_active=1`. The partial-unique index `idx_user_models_one_active ON user_models(user_id) WHERE is_active=1` enforces one active per user.

But here's a concern: if there's an existing active row at a *different* version, the demote sets it to 0. Good. Then upsert sets the current version active. Good. No conflict. This looks correct.

Let me now examine the `get` method in AppConfig models more closely for a real bug. There's a subtle issue: `__getitem__` returns `self.get(key)` then checks `if val is None and key not in model_fields: raise KeyError`. But what if a top-level field genuinely has value None? Then it returns None correctly. What if a nested leaf is found and is None? Returns None ? fine. Let me look for something more concrete.

Let me reexamine `query_play_segments`. It uses a single cursor and iterates `rows` while issuing nested `cursor.execute` for events on the SAME cursor inside the loop. After `cursor.execute("SELECT * FROM events...")`, the original `rows` was already fully fetched via `.fetchall()`, so re-using the cursor is fine. Not a bug.

Let me look at the `_unknown_key_paths` function for a real bug with `Optional[...]` annotations. `fields[key].annotation` for `relevance: ValidationRelevanceConfig` is the class. But for nested Optional models like `FusionConfig`'s fields, the annotation could be `Optional[X]` which is not a `type`. The check `isinstance(ann, type) and issubclass(ann, BaseModel)` guards that. But this means a nested section declared as `Optional[SomeModel]` would NOT recurse and unknown keys inside it would be missed ? minor, not a crash. Let me check `IndexingConfig.tracknet/wasb/fusion` ? those are direct types, not Optional, so they recurse. Fine.

Now let me look at a genuinely suspicious area ? `set_job_waiting` uses `COALESCE(?, progress)` to keep progress if None passed. Fine.

Let me check `extract_video_chunk` ? the `-ss` before `-i` does input seeking which is fast but the `-t` duration. With `-ss` before `-i` and stream copy, this is a known ffmpeg behavior. Not a logic bug per se.

Let me look more carefully at the geometry `calculate_iou` `+1` convention combined with `get_velocity` ? and verify against the test expectations.

34. user (2026-06-18T01:29:13.088Z)

```
1       import math
2       from backend.utils.geometry import get_center, get_distance, get_velocity,
get_velocity_normalised, calculate_iou
3
4       def test_get_center():
5           bbox = (0.0, 0.0, 10.0, 10.0)
6           assert get_center(bbox) == (5.0, 5.0)
7
8           bbox2 = (10.0, 20.0, 30.0, 40.0)
9           assert get_center(bbox2) == (20.0, 30.0)
10
11      def test_get_distance():
12          p1 = (0.0, 0.0)
13          p2 = (3.0, 4.0)
14          assert math.isclose(get_distance(p1, p2), 5.0)
15
16      def test_get_velocity():
17          bbox1 = (0.0, 0.0, 10.0, 10.0)
18          bbox2 = (3.0, 4.0, 13.0, 14.0)
19          # Centers: (5,5) and (8,9)
20          # Distance: sqrt(3^2 + 4^2) = 5
21          # FPS: 30
22          # Velocity = 5 * 30 = 150.0
23          assert math.isclose(get_velocity(bbox1, bbox2, 30.0), 150.0)
24
25      def test_get_velocity_normalised():
26          bbox1 = (0.0, 0.0, 10.0, 10.0)
27          bbox2 = (3.0, 4.0, 13.0, 14.0)
28          # raw velocity = 150.0
29          # normalized with width 1000 = 150.0 / 1000.0 = 0.15
30          assert math.isclose(get_velocity_normalised(bbox1, bbox2, 30.0, 1000.0), 0.15)
31
32          # Check fallback when width <= 0
33          assert math.isclose(get_velocity_normalised(bbox1, bbox2, 30.0, 0.0), 150.0)
34
35      def test_calculate_iou():
36          boxA = (0.0, 0.0, 10.0, 10.0)
37          boxB = (0.0, 0.0, 10.0, 10.0)
38          assert math.isclose(calculate_iou(boxA, boxB), 1.0)
39
40          boxC = (100.0, 100.0, 110.0, 110.0)
41          assert calculate_iou(boxA, boxC) == 0.0
42
43          boxD = (5.0, 5.0, 15.0, 15.0)
```

```
44          # intersection: x=[5,10], y=[5,10] => area = 5 * 5 = 25 (Wait, the iou function adds
+1 to width and height)
45          # Let's test actual output based on the specific IoU logic (+1 for bounds)
46          iou = calculate_iou(boxA, boxD)
47          assert iou > 0.0 and iou < 1.0
48
```

35. assistant (2026-06-18T01:29:20.938Z)

The `+1` IoU convention is intentional/tested. Not a bug.

Let me now reconsider the `claim_due_job` more carefully for a real race/correctness bug. The compare-and-set updates `WHERE id=? AND status=?`. The `_connect()` wraps in `with conn:` (transaction) AND the method also does `conn.commit()`. But importantly, the SELECT and UPDATE happen in the same connection. With WAL and `busy_timeout`, two workers could both SELECT the same row before either UPDATEs. Then both attempt the UPDATE. The first wins (rowcount=1), the second's UPDATE matches `status='queued'` but the row is now 'running', so rowcount=0 ? returns None. This is correctly handled. Not a bug.

Let me look at `seconds_until_next_due`. It computes `(julianday(next_poll_at) - julianday('now')) * 86400.0`. `next_poll_at` was set via `datetime('now', '+N seconds')` which is UTC. `julianday('now')` is UTC. Consistent. Fine.

Let me re-examine the telemetry `_iso` uses `time.localtime` but `started` in meta also uses localtime. Consistent for display.

Now let me look at a more subtle one in `RunTelemetry.snapshot`: fps computed only when `now > self._last_t`. And `self._last_done, self._last_t = done, now` is set INSIDE the `if done is not None:` block but only when... let me re-read.

36. user (2026-06-18T01:29:22.459Z)

```
148         def snapshot(self, done: Optional[int] = None, total: Optional[int] = None,
149                      phase: Optional[str] = None, **extra: Any) -> Dict[str, Any]:
150             """Record a velocity + health sample; ``fps`` is derived from the previous
snapshot."""
151             now = self._clock()
152             rec: Dict[str, Any] = {"t": self._iso(now), "elapsed_s": round(now - self._t0,
1)}
153             if phase is not None:
154                 rec["phase"] = phase
155             if done is not None:
156                 rec["done"] = done
157                 if total:
158                     rec["total"] = total
159                     rec["pct"] = round(100.0 * done / total, 1)
160                 if self._last_done is not None and self._last_t is not None and now >
self._last_t:
161                     rec["fps"] = round((done - self._last_done) / (now - self._last_t), 1)
162                 self._last_done, self._last_t = done, now
163             rec.update(extra)
164             self._attach_health(rec)
165             self._append(rec)
166             return rec
167
```

37. assistant (2026-06-18T01:29:38.157Z)

This is fine. `self._last_done/_last_t` update only when `done is not None`. fps computed against previous done snapshot.

Let me reconsider `_unknown_key_paths` for a genuine crash. `fields[key].annotation` ? for a field like `allowed_upload_extensions: List[str]`, the annotation is `List[str]`. If the file provides a dict for that key (mismatched type), `isinstance(val, dict)` True, then

`isinstance(ann, type)` ? `List[str]` is not a `type` instance (it's a `typing` generic), so False. No crash. Good.

But what about `relevance` field? annotation is `ValidationRelevanceConfig` (a type, subclass of BaseModel). If file provides `relevance: {...}` it recurses. Good. If file provides `relevance: "notadict"` (string), `isinstance(val, dict)` False, skip. No crash.

Now let me examine the `merged` dict in `load_config`. `merged = {**_get_builtin_defaults(), **file_data}`. This is a SHALLOW merge. So if `file_data` has `{"indexing": {"use_gpu": false}}`, the entire `indexing` section from defaults is REPLACED by `{"use_gpu": false}`, losing all other indexing defaults. But Pydantic re-validates `merged` and `IndexingConfig` re-applies its own field defaults for missing keys. So `indexing.motion_threshold` etc. get defaults back. So shallow merge is fine because the models have defaults. Not a bug.

Let me look at something I want to verify: the `add_play_segment` returns `cursor.lastrowid`. With the `_connect` context manager doing `with conn:` then `conn.commit()` then `return cursor.lastrowid` ? but the `return` is inside the `with self._connect() as conn:` block. After the inner code, the `with conn:` commits, then `finally: conn.close()`. `cursor.lastrowid` is read before close. Fine.

Now let me dig into a potential real issue. Look at `claim_due_job`: it returns `self.get_job(row["id"])` AFTER the `with self._connect()` block closes the connection. `get_job` opens a NEW connection. Between the UPDATE commit and the get_job, another worker could theoretically transition the job, but the claim already succeeded. Minor, not a bug.

Let me now check the `_connect` contextmanager interaction with methods that ALSO call `conn.commit()` explicitly. The `with conn:` block commits on exit. The explicit `conn.commit()` inside is redundant but harmless. However, there's a subtle issue: in `claim_due_job`, after `conn.commit()`, it checks `cur.rowcount`. Fine.

Let me look broader. Let me check the `execution_policy.from_dict` for a crash when `mode` is an invalid string.

`WaitMode(kw["mode"])` ? if `kw["mode"]` is an unknown string like `"bogus"`, `WaitMode("bogus")` raises `ValueError`. This is reachable if a job's stored policy JSON has a bad mode. But that's a deserialization of trusted data. Let me check where `from_dict` is called.

38. user (2026-06-18T01:29:39.182Z)

```
tests\test_execution_policy.py:52:    assert ExecutionPolicy.from_dict(d) == p
tests\test_execution_policy.py:54:    assert ExecutionPolicy.from_dict({"mode": "block", "junk":
1}).mode is WaitMode.BLOCK
backend\infrastructure\execution_policy.py:61:    def from_dict(cls, d: dict) ->
"ExecutionPolicy":
tests\test_experiment.py:59:    assert Variant.from_dict(v.to_dict()) == v
tests\test_experiment.py:65:def test_from_dict_ignores_unknown_keys():
tests\test_experiment.py:66:    v = Variant.from_dict({"name": "x", "min_crossings": 1, "bogus":
99})
backend\pipeline\personalization\serving.py:17:    ``LogisticRegression`` via ``from_dict``
(None when the user has no active model or it is a
backend\pipeline\personalization\serving.py:91:    ``from_dict``) when the user has an active
model that carries one; None when there is no
backend\pipeline\personalization\serving.py:103:    return LogisticRegression.from_dict(blob)
backend\eval\ablation.py:226:    return experiment.Variant.from_dict({**full.to_dict(),
**overrides,
backend\eval\classifier.py:98:    def from_dict(cls, d: dict) -> "LogisticRegression":
backend\eval\classifier.py:114:            return cls.from_dict(json.load(f))
backend\eval\experiment.py:144:    def from_dict(cls, d: dict) -> "Variant":
tests\test_training_gen0.py:95:    clf = LogisticRegression.from_dict(payload)
tests\test_variant_tuning.py:36:    assert Variant.from_dict(d).tune_threshold is True
```

39. assistant (2026-06-18T01:30:05.933Z)

`ExecutionPolicy.from_dict` isn't called on the DB job-policy path (the worker reads the dict

directly). Not a strong reachable bug.

Let me now look at the worker that consumes the job model to understand the contract for `claim_due_job`'s ordering and whether there's a real defect. Actually, let me step back and look harder at the `LogisticRegression.from_dict` and `serving.py` since serving loads `classifier_json` from the DB. But that's outside my area (`backend/pipeline/personalization/`).

Let me focus on my assigned area and look one more time at the migration ladder for a real defect. The migrations run inside a single `with self._connect() as conn:` in `_init_db`. Each `if version < N` block runs `ALTER TABLE`. Consider: a fresh DB has `user_version=0`. All blocks run sequentially in one transaction. The `version` variable is read ONCE at the start (0) and never re-read. So all `if version < N` are evaluated against the original version. Correct ? that's the intended ladder behavior.

But there's a subtle issue: `_init_db` does the migrations inside `with self._connect() as conn:` which wraps `with conn:` (a transaction). DDL statements like `ALTER TABLE` and `CREATE TABLE` in SQLite... In Python's sqlite3 with default isolation, DDL statements implicitly commit. Actually in SQLite, DDL can run in transactions. The explicit `conn.commit()` at the end is fine.

Let me check a real concern: `_get_connection` sets `PRAGMA journal_mode = WAL` inside the try. But `_init_db` is called from `__init__` via `self._connect()` ? `_get_connection()`. Fine.

Let me look at whether there's an issue with the order of operations in `claim_due_job`'s `ORDER BY COALESCE(next_poll_at, created_at)`. For a `queued` job, `next_poll_at` is NULL ? uses `created_at`. For a `waiting` job ? uses `next_poll_at`. Now consider a queued job created at T0 and a waiting job due at T0+1min. The waiting job's next_poll_at > created_at of queued. So queued job is picked first (earlier). That seems intended (earliest-due first). But wait ? both are "runnable". If a waiting job became due EARLIER (next_poll_at < queued's created_at), it'd be picked first. That's reasonable. Not a bug.

Let me now look at the `inactivity_min_density` default mismatch. In `IndexingConfig`, `inactivity_min_density: float = 0.34`. The docstring for R4 mentions "min_density=0.30". And the comment says default 0.34. Let me check the trajectory_to_action_windows signature default: `inactivity_min_density: float = 0.34`. Consistent. Not a bug.

Let me check a subtle thing: `inactivity_rally_thresh` default. In `IndexingConfig` it's `0.08`. The R4 doc says measured win at `rally_thresh=0.20`. But the default is OFF (`window_inactivity_end=0.0`), so the thresh default doesn't matter unless enabled. Not a bug.

Let me look more carefully at the `features()` function for a divide concern. `spd_mean = uniform_filter1d(speed, size) / np.maximum(sm, 1e-9)`. `sm` is the windowed mean of `spd_mask`. This is the mean speed over frames where speed was computed. Looks intentional. `cross_rate = uniform_filter1d(cross, size) * fps`. The `uniform_filter1d` gives mean of cross over window (crossings per frame), times fps ? crossings per second. Correct.

Now `spd_max = maximum_filter1d(speed, size, mode="constant")`. Fine.

Let me reconsider the `idx = np.arange(0, N, max(1, int(stride_sec * fps)))` and `idx/fps` returns times. Fine.

Let me now examine an important potential bug in `build_frame_arrays`: the speed loop uses `prev` to track the previous VISIBLE frame. But `prev` is set to `f` at the end of each iteration regardless of the gap. So after a long occlusion gap (df > max_gap), speed[f]=0 (not computed) but `prev=f` updates. Correct.

Wait ? there's a subtle issue. The loop iterates `for p in pts:` and uses `f = p.frame`. It checks `if not (0 <= f < N and vis[f]): continue`. But `vis[f]` was set in the first loop only for visible points with `p.x >= 0`. So if a point is visible but `p.x < 0` (the invisible sentinel `-1`), `vis[f]` stays 0, and this point is skipped in the speed loop. But `prev` is NOT updated for skipped points (the `continue` happens before `prev = f`). Good.

But there's a duplicate-frame concern: `pts` is sorted by frame, and multiple points could share the same frame. The first loop overwrites x[frame]/y[frame] for each. In the speed loop, if two points have the same frame `f`, then `df = f - prev = 0` on the second ? `0 < df` is False ?

speed not computed, but `prev = f` updates to same value. Fine.

I'm not finding strong bugs in features. Let me look at one more area ? the `paths.py` `resolve_under_root`. `os.path.commonpath([rp, rr]) == rr`. Consider `root="/data/videos"` and `path="/data/videos2/x.mp4"`. `realpath` ? `/data/videos2/x.mp4`. `commonpath(["/data/videos2/x.mp4", "/data/videos"])` = `/data`. `/data != /data/videos` ? not contained ? raises. Correct (videos2 is not under videos). Good, this is the classic prefix bug correctly avoided by commonpath.

Let me double-check `validate_input_path` and `safe_output_filename` against their tests for completeness, then move on.

40. user (2026-06-18T01:30:07.055Z)

```
1       """Path-safety at the API boundary: output_filename write-traversal + input
containment."""
2
3       import pytest
4
5       from backend.utils.paths import safe_output_filename, validate_input_path,
resolve_under_root
6
7
8       @pytest.mark.parametrize("bad", [
9           "", "   ", "../evil.mp4", "a/b.mp4", "a\\b.mp4", "/etc/passwd",
10          "..", ".", "sub/../../x.mp4",
11      ])
12      def test_safe_output_filename_rejects(bad):
13          with pytest.raises(ValueError):
14              safe_output_filename(bad)
15
16
17      @pytest.mark.parametrize("ok", ["highlights.mp4", "my_reel-1.mp4", "out.MP4"])
18      def test_safe_output_filename_accepts_bare(ok):
19          assert safe_output_filename(ok) == ok
20
21
22      def test_validate_input_path_rejects_null_byte():
23          with pytest.raises(ValueError):
24              validate_input_path("a\x00b.mp4")
25
26
27      def test_validate_input_path_no_root_is_permissive(tmp_path):
28          p = tmp_path / "vid.mp4"
29          p.write_text("x")
30          validate_input_path(str(p))  # no allowed_root -> only null-byte check; must not
raise
31
32
33      def test_validate_input_path_enforces_root(tmp_path):
34          root = tmp_path / "uploads"
35          root.mkdir()
36          inside = root / "v.mp4"
37          inside.write_text("x")
38          validate_input_path(str(inside), allowed_root=str(root))  # contained -> ok
39          outside = tmp_path / "other.mp4"
40          outside.write_text("x")
41          with pytest.raises(ValueError):
42              validate_input_path(str(outside), allowed_root=str(root))
43
44
45      def test_resolve_under_root_blocks_traversal(tmp_path):
46          root = tmp_path / "r"
47          root.mkdir()
48          with pytest.raises(ValueError):
```

```
            resolve_under_root(str(root / ".." / "escape.mp4"), str(root))

```

41. assistant (2026-06-18T01:30:10.630Z)

Path safety is solid. Let me now look at the `logging_setup.py` which I haven't read, and reconsider the database `get` config shim once more.

42. user (2026-06-18T01:30:10.855Z)

```
    """Central logging setup for the indexer's CLI/run entrypoints.

    One place to (a) install the root log format every entrypoint shares and (b) tame
    the third-party HTTP/RPC client libraries that log one INFO line per request. On the
    Gemini path (``google-genai`` -> ``httpx``/``grpc``/``google.auth``) that is dozens of
    lines per run, which buries our own ``[rally]``/``[compile]``/``[CACHE]`` output in
    ``run.log`` ? the very thing the owner eyeballs during manual QA.

    The suppression is config-gated (``logging.verbose_http``) so request-flow debugging can
    restore the chatter without code edits, and it only ever touches the named third-party
    loggers ? our own ``backend.*`` loggers are left at the root level untouched.
    """

    from __future__ import annotations

    import logging

    #: Third-party client loggers that emit per-request INFO chatter. Raised to WARNING by
    #: default so only genuine warnings/errors from them reach run.log.
    NOISY_HTTP_LOGGERS = (
        "httpx",
        "httpcore",
        "urllib3",
        "google_genai",   # google-genai SDK (underscore form)
        "google.genai",   # dotted form, depending on SDK version
        "google.auth",
        "grpc",
    )

    _LOG_FORMAT = "%(asctime)s [%(name)s] %(levelname)s: %(message)s"
    _DATE_FORMAT = "%H:%M:%S"


    def setup_logging(verbose_http: bool = False, level: int = logging.INFO) -> None:
        """Configure root logging and (un)suppress noisy third-party HTTP loggers.

        Idempotent and safe to call more than once: ``basicConfig`` installs the root
        handler/format only the first time, while the third-party levels are (re)applied on
        every call ? so an entrypoint can call this early with the quiet default and again
        after config load to honour ``logging.verbose_http``.

        Args:
            verbose_http: when True, restore full per-request chatter (levels set to
                NOTSET so they inherit the root level); when False (default), raise the
                noisy loggers to WARNING.
            level: root logger level for our own ``backend.*`` logs (default INFO).
        """
        logging.basicConfig(level=level, format=_LOG_FORMAT, datefmt=_DATE_FORMAT)
        # basicConfig is a no-op once the root logger already has a handler (a prior
        # basicConfig call, a notebook, or a test runner) ? set the level explicitly so the
        # `level` argument always takes effect.
        logging.getLogger().setLevel(level)

        http_level = logging.NOTSET if verbose_http else logging.WARNING
        for name in NOISY_HTTP_LOGGERS:
```

```
56              logging.getLogger(name).setLevel(http_level)
57
```

43. assistant (2026-06-18T01:30:27.486Z)

logging_setup is clean.

Now let me reconsider the AppConfig `get()` shim very carefully, because it's used widely by validators/segmenters and a subtle bug there is high-impact.

```python
def get(self, key: str, default: Any = None) -> Any:
    _MISSING = object()
    val: Any = _MISSING
    if key in type(self).model_fields or hasattr(self, key):
        val = getattr(self, key, _MISSING)
    ...
```

Consider a top-level extra key stored via `extra="allow"`. E.g. config.json has `{"foo": 123}` and `foo` is not a model field. With `extra="allow"`, Pydantic stores it as an attribute. So `hasattr(self, "foo")` ? True. `getattr(self, "foo")` ? 123. Returns 123. Good.

Now consider the nested-leaf fallback: `for fname in type(self).model_fields: section = getattr(self, fname, None); if isinstance(section, BaseModel) and hasattr(section, key)`. So `config.get("use_gpu")` finds it in `indexing`. Good. But what about a leaf key that exists in MULTIPLE sections, e.g. `timeout_sec` exists in both `tracknet` and `wasb` ? but those are nested under `indexing`, not top-level sections. The top-level sections are `validation, indexing, output, stitching, security, logging`. `timeout_sec` is not a direct attribute of any of those. `hasattr(indexing, "timeout_sec")` ? False (it's under indexing.tracknet). So nested-leaf only goes one level deep. That matches the documented "find a leaf key inside any nested section" ? one level. Existing callers presumably use it that way. Not a crash.

But here's a real concern: `velocity_threshold` exists in BOTH `indexing.tracknet` and... no, those are second-level. At top-level sections, is there any duplicate leaf? `IndexingConfig` has many leaves; other sections too. `min_segment_duration` only in indexing. The fallback returns the FIRST section (in model_fields order) that has the key. If two top-level sections shared a leaf name, it'd be ambiguous but deterministic. Not a present bug.

Let me look at something potentially real: `__getitem__`.

```python
def __getitem__(self, key):
    val = self.get(key)
    if val is None and key not in type(self).model_fields:
        raise KeyError(key)
    return val
```

If `key` is a nested leaf like `config["use_gpu"]` where use_gpu is `True`, `get` returns True, not None ? returns True. But if `config["allowed_video_root"]` where the value is genuinely `None` (default) and the key is a nested leaf (under `security`), then `val is None` AND `key not in model_fields` (allowed_video_root is not a top-level field) ? raises KeyError! That's a real wrong behavior: a config leaf that legitimately defaults to None becomes a KeyError on subscript access instead of returning None.

Let me verify: `allowed_video_root` is in `SecurityConfig`, default `None`.
`config["allowed_video_root"]`:
- `get("allowed_video_root")`: not a top-level field, `hasattr(self, "allowed_video_root")`? AppConfig doesn't have that attr directly ? val stays _MISSING. Then nested loop: `security` section has `allowed_video_root` ? val = None. Returns None.
- Back in `__getitem__`: `val is None` True, `"allowed_video_root" not in model_fields` True ? raises KeyError.

So `config["allowed_video_root"]` raises KeyError even though the key is valid and just None-

valued. Is this reachable? Let me check whether any caller uses subscript access for None-valued leaves. Let me search.

44. user (2026-06-18T01:30:28.643Z)

```
backend\annotations\automated_provider.py:45:        "annotation_registry.get('auto',
labeler=<fn>, lineage='<model-family>'). "
backend\main.py:55:@app.get("/", response_class=HTMLResponse)
backend\main.py:119:@app.get("/api/config")
backend\main.py:262:        player_pool=job.get("player_pool"),
backend\main.py:263:        max_frames=job.get("max_frames"),
backend\main.py:264:        segmenter=job.get("segmenter"),
backend\main.py:282:        if result.get("video_id"):
backend\main.py:380:@app.get("/api/jobs/{job_id}")
backend\main.py:387:    result = job.get("result")
backend\main.py:391:        "progress": job.get("progress"),
backend\main.py:392:        "video_id": job.get("video_id"),
backend\main.py:393:        "error": job.get("error"),
backend\main.py:395:        "reports": (result or {}).get("reports"),
backend\main.py:396:        "created_at": job.get("created_at"),
backend\main.py:397:        "started_at": job.get("started_at"),
backend\main.py:398:        "finished_at": job.get("finished_at"),
backend\main.py:537:@app.get("/api/highlights/{video_id}/{filename}")
backend\main.py:599:        meta = s.get("metadata") or {}
backend\main.py:600:        sc = meta.get("shot_count") if isinstance(meta, dict) else None
backend\main.py:688:            threshold = cfg.get("indexing", {}).get("motion_threshold",
0.08)  # type: ignore[attr-defined]
backend\main.py:856:                print(f"[REPORT] Technical report :
{paths.get('technical_md')}")
backend\main.py:857:                print(f"[REPORT] Customer summary :
{paths.get('customer_md')}")
backend\main.py:858:                print(f"[REPORT] Machine JSON     : {paths.get('json')}")
backend\reporting\harvest.py:41:    ffmeta = (val_context or {}).get("ffprobe_meta") or {}
backend\reporting\harvest.py:42:    streams = ffmeta.get("streams") or []
backend\reporting\harvest.py:43:    fmt = ffmeta.get("format") or {}
backend\reporting\harvest.py:46:        w, h = s.get("width"), s.get("height")
backend\reporting\harvest.py:50:        fps_str = s.get("r_frame_rate", "0/1")
backend\reporting\harvest.py:58:        if s.get("codec_name"):
backend\reporting\harvest.py:59:            meta["container"] = fmt.get("format_name") or
s.get("codec_name")
backend\reporting\harvest.py:60:    if fmt.get("duration"):
backend\reporting\harvest.py:65:    if fmt.get("format_name"):
backend\reporting\harvest.py:66:        meta["container"] = fmt.get("format_name")
backend\reporting\harvest.py:74:    audio_streams = [s for s in streams if (s or
{}).get("codec_type") == "audio"]
backend\reporting\harvest.py:106:        total_play = round(sum(float(s.get("duration", 0.0))
for s in play_segments), 1)
backend\reporting\harvest.py:109:            detector_tag = (play_segments[0].get("metadata") or
{}).get("detector")
backend\reporting\harvest.py:144:    provider = (config.get("ai_provider") or "gemini")
backend\reporting\harvest.py:152:            "model": config.get("ai_model") if ai_based else
None,
backend\reporting\harvest.py:156:            "oversize_clips_skipped": int((run_signals or
{}).get("oversize_clips_skipped", 0)) if ai_based else None,
backend\reporting\harvest.py:164:    total = round(sum(float(s.get("duration", 0.0)) for s in
play_segments), 1)

[Showing results with pagination = limit: 40]
```

45. assistant (2026-06-18T01:30:32.180Z)

The callers mostly use `config.get("section", {}).get(...)` which is the `.get()` method (returns the default on missing), not `__getitem__`. The `.get()` path doesn't have the KeyError bug. Let me confirm whether `__getitem__` (subscript) is actually used anywhere on the config with a None-valued leaf. Let me search for subscript usage on config objects specifically.

46. user (2026-06-18T01:30:33.716Z)

```
backend\pipeline\personalization\serving.py:22:static ``idx_cfg["fusion"]`` ? it does NOT call
anything here. Wiring this resolver into that
backend\pipeline\personalization\serving.py:61:    ``idx_cfg["fusion"]``). The original
``served_cfg`` is never mutated.
backend\pipeline\detectors\wasb_infer.py:395:    frames_in = int(cfg["model"]["frames_in"])
backend\pipeline\detectors\wasb_infer.py:396:    input_wh = (int(cfg["model"]["inp_width"]),
int(cfg["model"]["inp_height"]))
backend\pipeline\detectors\wasb_infer.py:397:    output_wh = (int(cfg["model"]["out_width"]),
int(cfg["model"]["out_height"]))
tests\test_personalization_serving.py:16:# A representative served fusion config dict (mirrors
idx_cfg["fusion"] from FusionConfig).
```

47. user (2026-06-18T01:30:34.699Z)

```
1       """Basic tests for the typed configuration package.
2
3       These ensure:
4       - Defaults are sensible and come from the models.
5       - File overrides work.
6       - Old config files with extra keys are tolerated.
7       - save_default_config produces loadable output.
8       """
9       import json
10      import tempfile
11      import os
12
13
14      from backend.config import load_config, save_default_config, AppConfig
15
16
17      def test_load_default_config():
18          # Force load without the project's config.json so we test the model built-in
defaults
19          cfg = load_config("/nonexistent/path/that/does/not/exist/12345/config.json")
20          assert isinstance(cfg, AppConfig)
21          assert cfg.sport == "badminton"
22          assert cfg.indexing.default_segmenter == "yolo_hybrid"
23          assert cfg.validation.min_blur_threshold == 20.0  # updated default from review
24          assert cfg.indexing.ai_max_workers == 5
25
26
27      def test_logging_config_default_quiets_http():
28          """verbose_http defaults to False so run.log is quiet unless explicitly turned
up."""
29          cfg = load_config("/nonexistent/path/12345/config.json")
30          assert cfg.logging.verbose_http is False
31
32
33      def test_skip_ai_handoff_default_off():
34          """Fully-local mode is opt-in: skip_ai_handoff defaults False so the AI handoff
runs."""
35          cfg = load_config("/nonexistent/path/12345/config.json")
36          assert cfg.indexing.skip_ai_handoff is False
37
38
39      def test_skip_ai_handoff_file_override():
40          with tempfile.TemporaryDirectory() as td:
41              p = os.path.join(td, "skip.json")
42              with open(p, "w") as fh:
43                  json.dump({"indexing": {"skip_ai_handoff": True}}, fh)
44              cfg = load_config(p)
45              assert cfg.indexing.skip_ai_handoff is True
46
```

```
47
48      def test_logging_config_file_override():
49          with tempfile.TemporaryDirectory() as td:
50              p = os.path.join(td, "logging.json")
51              with open(p, "w") as fh:
52                  json.dump({"logging": {"verbose_http": True}}, fh)
53              cfg = load_config(p)
54              assert cfg.logging.verbose_http is True
55
56
57      def test_fusion_config_defaults_reproduce_control():
58          """The fusion block defaults must be no-ops: wasb substrate, no cues, no gating, no
mask.
59
60          Gate A (min_crossings) defaults to 0 = OFF: the #146 served-default flip was
REVERTED
61          (eval?serve ? the +0.074 was on the harness's recall-oriented PROPOSAL windowing; on
the
62          served COARSE windowing Gate A is neutral/negative, ?0.008, dropping real rallies).
See
63          docs/QUALITY_ITERATIONS.md and the served litmus in
tests/test_served_gate_a_regression.py.
64          """
65          cfg = load_config("/nonexistent/path/12345/config.json")
66          f = cfg.indexing.fusion
67          assert f.substrate == "wasb"
68          assert f.cue_flags == {}            # person cue OFF
69          assert f.min_crossings == 0         # Gate A OFF (default) ? #146 served flip
reverted
70          assert f.min_players == 0           # Gate B OFF
71          assert f.court_polygon is None      # no mask
72          assert f.velocity_threshold == 0.02
73
74
75      def test_fusion_config_gate_a_opt_in_via_min_crossings():
76          """Gate A is opt-in: setting min_crossings explicitly enables it (the eval-only
operating
77          point), while the served default stays OFF (#146 flip reverted)."""
78          with tempfile.TemporaryDirectory() as td:
79              p = os.path.join(td, "optin.json")
80              with open(p, "w") as fh:
81                  json.dump({"indexing": {"fusion": {"min_crossings": 2}}}, fh)
82              cfg = load_config(p)
83              assert cfg.indexing.fusion.min_crossings == 2   # Gate A ON only when explicitly
set
84
85
86      def test_served_default_segmenter_is_gemini():
87          """The shipped config.json served default is the prior detector (gemini) ? the #146
flip
88          to fusion_hybrid was REVERTED as a confirmed served regression (eval?serve)."""
89          repo_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
90          cfg = load_config(os.path.join(repo_root, "config.json"))
91          assert cfg.indexing.default_segmenter == "gemini"
92          # ...and the served fusion default keeps Gate A OFF (no served gating).
93          assert cfg.indexing.fusion.min_crossings == 0
94
95
96      def test_fusion_config_file_override():
97          with tempfile.TemporaryDirectory() as td:
98              p = os.path.join(td, "fusion.json")
99              with open(p, "w") as fh:
100                 json.dump({"indexing": {"fusion": {
101                     "substrate": "tracknet", "min_crossings": 2,
102                     "cue_flags": {"person": True}, "court_polygon": [[0.1, 0.1], [0.9, 0.1],
```

```
[0.5, 0.9]],
                }}}, fh)
            cfg = load_config(p)
            f = cfg.indexing.fusion
            assert f.substrate == "tracknet"
            assert f.min_crossings == 2
            assert f.cue_flags == {"person": True}
            assert len(f.court_polygon) == 3
            # An old config without a fusion block still loads (defaults applied).
            assert cfg.indexing.wasb.velocity_threshold == 0.02


    def test_file_override():
        with tempfile.TemporaryDirectory() as td:
            p = os.path.join(td, "override.json")
            data = {
                "indexing": {
                    "default_segmenter": "gemini",
                    "ai_max_workers": 3,
                },
                "validation": {
                    "min_blur_threshold": 42.0,
                },
            }
            with open(p, "w") as f:
                json.dump(data, f)

            cfg = load_config(p)
            assert cfg.indexing.default_segmenter == "gemini"
            assert cfg.indexing.ai_max_workers == 3
            assert cfg.validation.min_blur_threshold == 42.0
            # Unspecified keys fall back to defaults
            assert cfg.sport == "badminton"


    def test_extra_keys_tolerated():
        """Old or future keys in config.json must not break loading."""
        with tempfile.TemporaryDirectory() as td:
            p = os.path.join(td, "with_extra.json")
            data = {
                "sport": "badminton",
                "indexing": {"default_segmenter": "yolo_hybrid"},
                "some_future_key": "ignored_for_now",
                "validation": {"relevance": {"court_pixels_min_ratio": 0.1}},
            }
            with open(p, "w") as f:
                json.dump(data, f)

            cfg = load_config(p)
            assert cfg.indexing.default_segmenter == "yolo_hybrid"
            assert cfg.validation.relevance.court_pixels_min_ratio == 0.1


    def test_unknown_keys_warn_with_dotted_paths(tmp_path, caplog):
        """Typo'd keys must be LOUD: top-level extras are kept-but-unread and unknown
        section keys are silently dropped, so without this warning a typo just makes
        the knob a no-op (the audit's 'silently dropped config' class)."""
        p = tmp_path / "typo.json"
        p.write_text(json.dumps({
            "some_future_key": 1,                                  # top-level extra
            "indexing": {"velocty_threshold": 0.5},                # typo'd section key
            "validation": {"relevance": {"unknown_gate": True}},   # doubly nested
        }))
        import logging
        with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
```

```
            cfg = load_config(str(p))
        assert isinstance(cfg, AppConfig)  # still non-fatal
        joined = " ".join(r.message for r in caplog.records)
        assert "some_future_key" in joined
        assert "indexing.velocty_threshold" in joined
        assert "validation.relevance.unknown_gate" in joined


    def test_recognized_config_does_not_warn(tmp_path, caplog):
        p = tmp_path / "clean.json"
        p.write_text(json.dumps({
            "sport": "badminton",
            "indexing": {"default_segmenter": "gemini", "wasb": {"velocity_threshold":
0.02}},
        }))
        import logging
        with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
            load_config(str(p))
        assert not [r for r in caplog.records if "CONFIG WARNING" in r.message]


    def test_save_default_config_roundtrip():
        with tempfile.TemporaryDirectory() as td:
            p = os.path.join(td, "saved.json")
            saved = save_default_config(p)
            assert str(saved) == p
            assert os.path.exists(p)

            cfg = load_config(p)
            assert cfg.indexing.default_segmenter == "yolo_hybrid"
            assert cfg.validation.min_blur_threshold == 20.0


    def test_model_is_the_source_of_truth():
        """Changing a default in the model should be reflected without touching json."""
        # This is a documentation test more than anything.
        cfg = AppConfig()
        assert cfg.indexing.tracknet.velocity_threshold == 0.02


    def test_output_dir_field_default_and_override():
        """output_dir now lives on OutputConfig (was a write-only runtime hack), so
        /api/compile and the CLI resolve the same value. Default + file override + the
        runtime mutation that backend/main.py::main performs for --output-dir."""
        cfg = AppConfig()
        assert cfg.output.output_dir == "output"  # built-in default

        # The CLI override path: main() assigns args.output_dir onto the typed model.
        cfg.output.output_dir = "/tmp/custom_out"
        assert cfg.output.output_dir == "/tmp/custom_out"

        # File override is honored on load.
        with tempfile.TemporaryDirectory() as td:
            p = os.path.join(td, "out_override.json")
            with open(p, "w") as f:
                json.dump({"output": {"output_dir": "reels", "db_name": "x.db"}}, f)
            loaded = load_config(p)
            assert loaded.output.output_dir == "reels"
            assert loaded.output.db_name == "x.db"

    def test_config_shallow_section_replacement():
        """Positive test for Item 2 (config loader): shallow section replacement.

        File-provided 'indexing' section replaces the defaults 'indexing' section entirely
        (no deep merge of sub-keys), per loader's {**defaults, **file_data}.
```

```
231         """
232         with tempfile.TemporaryDirectory() as td:
233             p = os.path.join(td, "shallow.json")
234             with open(p, "w") as fh:
235                 json.dump({"indexing": {"default_segmenter": "motion"}}, fh)
236             cfg = load_config(p)
237             assert cfg.indexing.default_segmenter == "motion"
238             # Pydantic defaults fill the rest of the (replaced) section
239             assert cfg.indexing.ai_max_workers == 5
240 
241 
242     def test_config_unknown_key_paths_warning_and_tolerate():
243         """Positive test for Item 2: _unknown_key_paths warning for unknown top-level keys,
244         but tolerated (extra='allow'), and unknown section keys dropped silently.
245         """
246         with tempfile.TemporaryDirectory() as td:
247             p = os.path.join(td, "unknown.json")
248             with open(p, "w") as fh:
249                 json.dump({
250                     "indexing": {"default_segmenter": "motion"},
251                     "completely_unknown_top": "bar",
252                     "validation": {"unknown_in_section": 99}  # dropped
253                 }, fh)
254             cfg = load_config(p)
255             assert cfg.indexing.default_segmenter == "motion"
256             # tolerated, no crash
257 
258 
259     def test_get_returns_dict_for_nested_sections_legacy_compat():
260         """Regression: legacy consumers (validators/segmenters) do
261         ``config.get("validation", {}).get("min_resolution_width", ...)``. The dict-compat
262         .get must return nested SECTIONS as plain dicts (not Pydantic models), or that
263         chained access raises 'X object has no attribute get' on every real run.
264 
265         This is the original strong legacy test (restored per #160 review). It covers the
266         full set of footgun guards: dict sections, nested dicts, scalar leaves with
defaults,
267         missing key fallbacks, and leaf keys discoverable inside sections.
268         """
269         cfg = load_config("/nonexistent/123/config.json")
270         validation = cfg.get("validation", {})
271         assert isinstance(validation, dict)
272         assert validation.get("min_resolution_width") == 1280
273         # nested-of-nested is also dict-accessible
274         assert isinstance(validation.get("relevance"), dict)
275         assert validation["relevance"].get("court_pixels_min_ratio") == 0.05
276 
277         # scalar leaves are returned unchanged; a present scalar ignores the default arg;
278         # a missing key falls back to the default
279         assert cfg.get("ai_provider") == "gemini"
280         assert cfg.get("ai_provider", "ignored-default") == "gemini"
281         assert cfg.get("does_not_exist", "fallback") == "fallback"
282 
283         # leaf keys inside any section resolve via the dynamic section scan (not a hardcoded
284         # list), so a future top-level section's keys would be discoverable too
285         assert cfg.get("min_resolution_width") == 1280  # lives inside `validation`
286         assert cfg.get("begin_padding") == 0.5          # lives inside `stitching`
287 
288         # legacy chained access still works
289         assert cfg.get("indexing", {}).get("default_segmenter") == "yolo_hybrid"
290 
291 
292     def test_config_roundtrip_and_typed_reaches_pipeline(monkeypatch):
293         """Positive for Item 2: save_default_config + load_config round-trips;
294         typed AppConfig is what reaches pipeline code (e.g. main uses global_config as
```

```
AppConfig).
295         """
296         with tempfile.TemporaryDirectory() as td:
297             p = os.path.join(td, "roundtrip.json")
298             save_default_config(p)
299             cfg = load_config(p)
300             assert isinstance(cfg, AppConfig)
301             assert cfg.sport == "badminton"
302
303         # positive that typed is used in pipeline code (global_config is set at startup in
main();
304         # monkeypatch here so the test does not depend on server initialization)
305         import backend.main as main_module
306         from backend.main import get_config
307         monkeypatch.setattr(main_module, "global_config", cfg)
308         assert isinstance(get_config(), AppConfig)
309
```

48. assistant (2026-06-18T01:30:47.266Z)

The tests confirm `__getitem__` is only exercised for sections (which are non-None). The `__getitem__` None-leaf KeyError is real but appears to have no production caller using subscript on a None-valued leaf ? `wasb_infer.py` and `serving.py` use `cfg["model"]`/`idx_cfg["fusion"]` on plain dicts, not AppConfig. So this is a latent inconsistency, not a reachable bug for a maintainer right now. I'll set it low/drop it given the precision bar.

Let me look at one thing that could be a genuine reachable defect: in `query_play_segments`, `filters.get("duration_min")` ? if `duration_min` is `0.0` (a falsy but valid value), `if filters.get("duration_min"):` is False, so the filter is skipped. That's a minor "0 falsy" issue but `duration_min=0` is a no-op filter anyway (>= 0 matches all). Not a real bug.

Let me reconsider whether there's a real bug in the `_unknown_key_paths` recursion with Pydantic v2 `FieldInfo.annotation`. For nested models wrapped in defaults like `relevance: ValidationRelevanceConfig = Field(default_factory=...)`, `.annotation` is `ValidationRelevanceConfig`. Good. This works per the test.

Now let me look harder at the database `claim_due_job` for the actual ORDER BY issue with mixed NULL. Actually, let me reconsider `seconds_until_next_due`: it only considers `status='waiting'` jobs. But `claim_due_job` also claims `status='queued'` jobs. A queued job is due NOW. If the worker computes `seconds_until_next_due()` to decide its sleep and there are only queued jobs (no waiting), it returns None ? the worker might sleep indefinitely while a queued job sits unclaimed. Let me check the worker loop to see if this is a real bug.

49. user (2026-06-18T01:30:47.997Z)

```
No files found
```

50. user (2026-06-18T01:30:48.475Z)

```
No files found
```

51. user (2026-06-18T01:30:49.208Z)

```
Found 2 files
backend\infrastructure\database.py
backend\main.py
```

52. user (2026-06-18T01:31:00.356Z)

```
5:import threading
67:# Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
77:        _job_executor = ThreadPoolExecutor(max_workers=1, thread_name_prefix="jobworker")
242:    releases the thread, and re-claims the job when due (`claim_due_job`). No work to date
```

```
258:    re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
269:    """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording its
304:    while True:
305:        job = db_instance.claim_due_job()
322:        delay = db_instance.seconds_until_next_due()
336:    timer = threading.Timer(delay_sec, lambda: _get_executor().submit(_drain_due_jobs))
353:    worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
410:_uploads_lock = threading.Lock()
786:    swept = db_instance.fail_stale_jobs()
```

53. user (2026-06-18T01:31:04.898Z)

```
296         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
297
298         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
299         up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
300         a single-worker box keeps making progress with no central timer (the DB is the
clock).
301         Returns the number of jobs that reached a terminal/parked state this tick.
302         """
303         ran = 0
304         while True:
305             job = db_instance.claim_due_job()
306             if job is None:
307                 break
308             ran += 1
309             _run_claimed_job(job)
310         # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
311         # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
312         # on the next submit/drain. Best-effort ? never raises into the worker.
313         _schedule_next_drain_if_waiting()
314         return ran
315
316
317     def _schedule_next_drain_if_waiting() -> None:
318         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
319         self-scheduled job resumes with no further client submit. The drain itself re-checks
320         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
321         try:
322             delay = db_instance.seconds_until_next_due()
323         except Exception as e:  # pragma: no cover - defensive; never wedge the worker
324             logger.error(f"Could not compute next-due delay: {e}")
325             return
326         if delay is None:
327             return  # nothing waiting
328         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
329         _arm_wake_timer(max(0.0, min(delay, _MAX_DRAIN_WAKE_SEC)))
330
331
332     def _arm_wake_timer(delay_sec: float) -> None:
333         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
334         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
335         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
336         timer = threading.Timer(delay_sec, lambda: _get_executor().submit(_drain_due_jobs))
337         timer.daemon = True
338         timer.start()
```

```
339
340
341     #: A parked job further out than this is re-checked by a capped wake rather than one
long
342     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
343     _MAX_DRAIN_WAKE_SEC = 60.0
344
345
346     @app.post("/api/process", status_code=202)
347     def process_video(req: ProcessRequest):
348         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
349
350         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
351         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
352         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
353         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
354         """
355         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
356         try:
357             validate_input_path(req.video_path, allowed_root=allowed_root)
358         except ValueError as e:
359             raise HTTPException(status_code=400, detail=str(e)) from e
360         if not os.path.exists(req.video_path):
361             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
362         if req.segmenter and not segmenter_registry.get(req.segmenter):
363             available = list(segmenter_registry.list_available().keys())
364             raise HTTPException(
365                 status_code=400,
366                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
367             )
368
369         job_id = uuid.uuid4().hex
370         if not db_instance.create_job(job_id, req.video_path, req.segmenter,
```

54. assistant (2026-06-18T01:31:10.515Z)

I see ? `_drain_due_jobs` loops `claim_due_job()` until None (so queued jobs are always fully drained in the current tick), THEN schedules the wake only for waiting jobs. So `seconds_until_next_due` returning None when only queued jobs exist is fine, because queued jobs are already drained before that point. The drain is triggered on each submit. So no stuck-queued bug. Good design.

Let me now examine the `_run_claimed_job` and the executor concurrency. `max_workers=1`. Let me check `_drain_due_jobs` for the `JobWaiting` handling and whether a waiting job re-enters the drain loop infinitely. Let me read around 240-295.

55. user (2026-06-18T01:31:11.914Z)

```
230                 val_results=val_results, val_context=val_context,
231                 segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
232                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
233             )
234             if paths and isinstance(response, dict):
235                 response["reports"] = paths
236
237
238     class JobWaiting(Exception):
239         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
240         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
241         NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
242         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
```

```
date
243         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
244
245         The wiring is additive: the production segmenter path never raises this today (zero
246         behaviour change), so a job runs exactly as before. The hook is what a later slice
247         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
248         """
249
250         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
251             super().__init__(f"job parked for {delay_sec:.0f}s")
252             self.delay_sec = max(0.0, float(delay_sec))
253             self.progress = progress
254
255
256     def _job_to_request(job: Dict[str, Any]) -> ProcessRequest:
257         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
258         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
259         the original submit. The row stores exactly the ProcessRequest fields."""
260         return ProcessRequest(
261             video_path=job["video_path"],
262             player_pool=job.get("player_pool"),
263             max_frames=job.get("max_frames"),
264             segmenter=job.get("segmenter"),
265         )
266
267
268     def _run_claimed_job(job: Dict[str, Any]) -> None:
269         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
270         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
271
272         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
273         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
274         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
275         job self-scheduled and will be re-claimed when `next_poll_at` is due.
276         """
277         job_id = job["id"]
278         req = _job_to_request(job)
279         try:
280             result = _execute_processing(
281                 req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
282             if result.get("video_id"):
283                 db_instance.set_job_video_id(job_id, result["video_id"])
284             db_instance.mark_job_done(job_id, result)
285         except JobWaiting as w:
286             # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
287             logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
288             db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
289         except Exception as e:
290             logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
291             db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
292
293
294     def _drain_due_jobs() -> int:
295         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
```

56. assistant (2026-06-18T01:31:31.277Z)

This is well-designed. Now consider: a job parked with `delay_sec=0` (JobWaiting(0)). `set_job_waiting` sets `next_poll_at = datetime('now', '+0 seconds')` = now. In the SAME `_drain_due_jobs` while-loop, the next `claim_due_job()` will find this waiting job

(`next_poll_at <= CURRENT_TIMESTAMP`) and re-claim it immediately, re-running it. If the segmenter keeps raising JobWaiting(0), this is an infinite tight loop. But the docstring says production never raises JobWaiting today, so it's a latent future concern, not a current reachable bug. Out of scope for the precision bar.

I've now thoroughly covered my area. Let me do one final careful pass on the migration ladder and `upsert_user_model`'s rowid lookup with NULL user_id, and double-check the `add_video` UPSERT doesn't reset `user_id` (added in v4).

Looking at `add_video`:
```sql
INSERT INTO videos (id, filepath, status, validation_results) VALUES (?, ?, ?, ?)
ON CONFLICT(id) DO UPDATE SET filepath=..., status=..., validation_results=...
```
This doesn't touch `user_id` or `processed_at`. On conflict it preserves existing `user_id`. On insert, `user_id` defaults to NULL. Correct per the v4 comment ("NULL = local install").

Let me verify one concrete potential bug in `upsert_user_model`: the demote-then-upsert is NOT atomic against the unique index in one specific scenario. Consider an existing active row at `version=1` and we call `upsert_user_model(version=2, is_active=True)`. The demote sets version=1's `is_active=0` (matches `version != 2`). Then the INSERT for version=2 with is_active=1. No two active rows ? index satisfied.

Now consider calling with the SAME version that's already active: existing active row at version=1, call `upsert(version=1, is_active=True)`. Demote: `WHERE ... is_active=1 AND version != 1` ? matches nothing (the active row IS version 1). Then UPSERT version=1 ON CONFLICT updates is_active=1 (already 1). Fine, no conflict.

Now the dangerous one: existing active row at version=1, call `upsert(version=2, is_active=True)` where version=2 doesn't exist yet but version=1 active. Demote version=1 ? is_active=0. Insert version=2 active. Good.

But what if BOTH version=1 (active) AND version=2 (inactive) exist, and we call `upsert(version=2, is_active=True)`? Demote: `version != 2 AND is_active=1` ? demotes version=1. Then UPSERT version=2 ? ON CONFLICT sets is_active=1. One active. Good.

The logic holds. This is solid, audited code.

Let me do a final check on `_iso` timezone and `gpu_stats` parsing where `nvidia-smi` returns `[N/A]` for some fields ? `_num("[N/A]")` returns None. Then `temp, util, ... = vals` could have None. `throttled = bool(temp is not None and temp >= ...)`. Handles None. `if sm is not None and sm_max:` handles None/zero sm_max. Good.

I've completed a thorough, trace-based review of my assigned area (infrastructure DB + migration ladder, config models/loader, utils, features). The code shows clear evidence of multiple hardening sweeps. The only inconsistency I found ? `AppConfig.__getitem__` raising `KeyError` for a valid-but-None-valued nested leaf ? has no reachable production caller (subscript access on AppConfig with a None leaf is never used; all callers use `.get()` or subscript only on sections), so by the precision-over-recall bar it does not qualify as a real, reachable bug a maintainer would fix.

I'll return an empty candidates list, which the instructions explicitly state is a valid, respected answer.

57. user (2026-06-18T01:31:32.269Z)

Structured output provided successfully

58. assistant (2026-06-18T01:31:45.092Z)

I completed a trace-based adversarial review of my assigned area and am returning an empty candidates list ? no real, reachable defects met the precision bar.

Files reviewed in full:
- `backend/infrastructure/database.py` ? migration ladder (PRAGMA user_version v1?v5),

connection/transaction handling (`_connect`/`_get_connection`), async job store (`claim_due_job` compare-and-set, `seconds_until_next_due`, `set_job_waiting`, `fail_stale_jobs`), per-user model store (`upsert_user_model` demote-then-upsert active-row invariant).
- `backend/config/models.py` ? Pydantic models, `IndexingConfig` validators (chunk-step positivity), the legacy `get()`/`__getitem__` dict-compat shim.
- `backend/config/loader.py` ? `_unknown_key_paths` recursion, shallow-merge + re-validate.
- `backend/utils/hashing.py`, `run_telemetry.py`, `video.py`, `geometry.py`, `paths.py`, `logging_setup.py`.
- `backend/features/rally_seq.py` ? `build_frame_arrays` / `features` (fps-normalized speed, occlusion gap, windowing).
- `backend/infrastructure/execution_policy.py` ? `run_bounded` / `poll_until`.

What I verified and cleared (with the worker loop in `backend/main.py` and the relevant tests):
- `claim_due_job` ordering and compare-and-set race handling are correct; `seconds_until_next_due` returning `None` for queued-only is fine because `_drain_due_jobs` fully drains queued jobs before scheduling the waiting-only wake.
- `upsert_user_model`'s demote (`version != ?`) + UPSERT preserves the one-active-per-user partial-unique invariant across all version permutations.
- `add_video` UPSERT (not REPLACE) preserves child rows and the v4 `user_id`.
- Path-safety (`resolve_under_root` uses `commonpath`, avoiding the prefix bug), config shallow-merge (models re-supply defaults), and the IoU `+1` convention are all intentional and test-pinned.

One inconsistency I traced but deliberately did NOT report (fails the reachable-bug bar): `AppConfig.__getitem__` raises `KeyError` for a valid-but-`None`-valued nested leaf (e.g. `config["allowed_video_root"]`), because it conflates "missing" with "present-but-None". No production caller uses subscript access on `AppConfig` for a None leaf ? all callers use `.get()` (which returns the default correctly) or subscript only on non-None sections / plain dicts (`cfg["model"]`, `idx_cfg["fusion"]`) ? so it is latent, not reachable.

The code in this area carries clear evidence of prior hardening sweeps; an empty result is the honest outcome.